



PDF Download
3757915.pdf
04 April 2026
Total Citations: 1
Total Downloads: 309

Latest updates: <https://dl.acm.org/doi/10.1145/3757915>

RESEARCH-ARTICLE

Fault Localization from the Semantic Code Search Perspective

YIHAO QIN, National University of Defense Technology China, Changsha, Hunan, China

SHANGWEN WANG, National University of Defense Technology China, Changsha, Hunan, China

YAN LEI, Chongqing University, Chongqing, China

ZHUO ZHANG, Tianjin University, Tianjin, China

BO LIN, National University of Defense Technology China, Changsha, Hunan, China

XIN PENG, National University of Defense Technology China, Changsha, Hunan, China

[View all](#)

Open Access Support provided by:

National University of Defense Technology China

Chongqing University

Tianjin University

Published: 26 August 2025

Accepted: 27 July 2025

Revised: 03 June 2025

Received: 25 December 2024

[Citation in BibTeX format](#)

Fault Localization from the Semantic Code Search Perspective

YIHAO QIN, National University of Defense Technology, China

SHANGWEN WANG*, National University of Defense Technology, China

YAN LEI, Chongqing University, China

ZHUO ZHANG, Tianjin University, China

BO LIN, National University of Defense Technology, China

XIN PENG, National University of Defense Technology, China

JUN MA, National University of Defense Technology, China

LIQIAN CHEN, National University of Defense Technology, China

XIAOGUANG MAO, National University of Defense Technology, China

The software development process is characterized by an iterative cycle of continuous functionality implementation and debugging, essential for the enhancement of software quality and adaptability to changing requirements. This process incorporates two isolatedly studied tasks: Code Search (CS), which retrieves reference code from a code corpus to aid in code implementation, and Fault Localization (FL), which identifies code entities responsible for bugs within the software project to boost software debugging. The basic observation of this study is that these two tasks exhibit similarities since they both address search problems. Notably, CS techniques have demonstrated greater effectiveness than FL ones, possibly because of the precise semantic details of the required code offered by natural language queries, which are not readily accessible to FL methods. Drawing inspiration from this, we hypothesize that a fault localizer could achieve greater proficiency if semantic information about the buggy methods were made available. Based on this idea, we propose CosFL, an FL approach that decomposes the FL task into two steps: *query generation*, which describes the functionality of the problematic code in natural language, and *fault retrieval*, which uses CS to find program elements semantically related to the query, allowing for finishing the FL task from a CS perspective. Specifically, to depict the buggy functionalities and generate high-quality queries, CosFL extensively harnesses the code analysis, semantic comprehension, text generation, and decision-making capabilities of LLMs. Moreover, to enhance the accuracy of CS, CosFL captures varying levels of context information and employs a multi-granularity code search strategy, which facilitates a more precise identification of buggy methods from a holistic view. The evaluation on 835 real bugs from 23 Java projects shows that CosFL successfully localizes 324 bugs within Top-1, which significantly outperforms the state-of-the-art approaches by 26.6%-57.3%. The ablation study and sensitivity analysis further validate the importance of different components and the robustness of CosFL across different backend models.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

*Corresponding author.

Authors' Contact Information: Yihao Qin, National University of Defense Technology, Changsha, China, yihaoqin@nudt.edu.cn; Shangwen Wang, wangshangwen13@nudt.edu.cn, National University of Defense Technology, Changsha, China; Yan Lei, Chongqing University, Chongqing, China, yanlei@cqu.edu.cn; Zhuo Zhang, Tianjin University, Tianjin, China, zz8477@126.com; Bo Lin, National University of Defense Technology, Changsha, China, linbo19@nudt.edu.cn; Xin Peng, National University of Defense Technology, Changsha, China, xinpeng@nudt.edu.cn; Jun Ma, National University of Defense Technology, Changsha, China, majun@nudt.edu.cn; Liqian Chen, National University of Defense Technology, Changsha, China, lqchen@nudt.edu.cn; Xiaoguang Mao, National University of Defense Technology, Changsha, China, xgmao@nudt.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s).

ACM 1557-7392/2025/8-ART

<https://doi.org/10.1145/3757915>

Additional Key Words and Phrases: Fault localization, Code search, Language models, Debugging

1 INTRODUCTION

The software development process is an iterative cycle of continuous functionality implementation and debugging [24]. During the implementation phase, programmers write code that meets specific requirements, but may simultaneously introduce bugs into the software system. In the debugging phase, developers search for the root causes through error signals and fix those program behaviors that do not meet expectations. This iterative process is crucial for continuously improving software quality and flexibly adapting to changing requirements. In the literature, two types of techniques are widely studied to facilitate development activities in this cycle, one for each phase. The first is **Code Search** (CS) [47] during the implementation phase, which aims to assist developers in reusing specific code snippets from open source repositories, rather than “reinventing the wheel”. The second is **Fault Localization** (FL) [59] during the debugging phase, which aims to identify the buggy program elements within the entire software system, and thus speed up the process of combating the bugs.

During the years, these two types of techniques have generally been studied in isolation, with few existing studies attempting to associate them. The only obvious commonality we can identify between these two types of techniques is the trend for both to leverage the strengths of advanced deep learning techniques in their technical development. Specifically, owing to the advancements in representation learning [53], CS techniques have evolved from traditional keyword matching to learning the semantic correlation between the requirements and code snippets. Similarly, FL techniques have also evolved from coverage-based program spectrum [3] to representation learning [55], or even empowered by latest Large Language Models (LLMs) [23]. Despite their shared trend, there is a significant discrepancy in the effectiveness of these two types of techniques. Experiments conducted on domain-specific benchmarks have shown that state-of-the-art CS techniques [56] can accurately identify the required code snippets for approximately 80% of the total requirements. In contrast, the latest FL technique [23] can only locate the buggy program entities at the top position for around 30% of the cases.

To boost the effectiveness of FL, the basic observation of this study is that the tasks of code search and fault localization inherently share certain similarities, as they both solve a search problem. In practical terms, CS takes requirements described in natural language queries as input and retrieves code from a large repository that aligns with these intentions. In this context, the queries contain rich semantic information about the required functionalities, allowing for the accurate identification of the necessary code [43]. On the other hand, FL takes a buggy project as input and identifies the program entities responsible for the bug. While FL has access to readily available information such as the source code, test suite, and failure information, it lacks explicit semantic information about the functionality of the buggy code. To address this shortfall, researchers have delved into leveraging advanced deep learning techniques, including coverage-based graph representation learning [34] and function call-based code navigation [23]. Despite these efforts, there remains ample room for enhancements in the efficacy of current fault localization techniques as aforementioned. Considering the differences in performance and inputs between these two types of techniques, we postulate that a fault localizer may perform better if it is provided with semantic information about the functionality of the buggy code. Consequently, the key idea of this study is to **equip the fault localizer with a natural language query that elucidates the functionality of the erroneous code, enabling fault localization to be performed in a code search fashion.**

Based on this idea, we propose CosFL, a **COde Search inspired Fault Localization** approach that localizes bugs at the method level. As illustrated in Figure 1, CosFL decomposes the fault localization task into two steps, namely, *query generation* and *fault retrieval*. In the first step, a natural language query is generated to describe the functionality of the possible problematic code, while in the second step, CS is employed to match suspicious program elements semantically related to the query from the entire project. As shown, this pipeline differs significantly from the workflows of existing representation learning-based FL [34] and code navigation-based

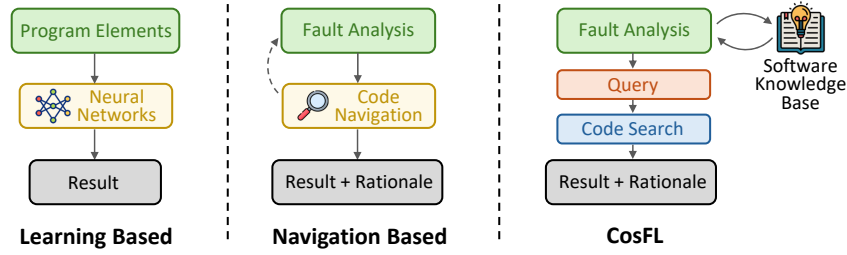


Fig. 1. Illustration of CosFL and other FL pipelines. The rationale is generated by LLMs.

FL [23], thus exploring the feasibility of a new fashion for FL. Intuitively, our pipeline faces two primary challenges: first, generating high-quality queries, as this information is not readily available and requires deduction through intricate code logic; and second, precisely pinpointing the buggy code, as the accuracy of code search still encounters challenges, particularly when different code segments within the same project exhibit semantic overlaps. To address the first challenge, we leverage the capabilities of LLMs for their promising abilities towards code analysis [61], semantic understanding [35], text generation [5], as well as autonomous decision-making [20]. In particular, we first construct a software knowledge base for the target project by amalgamating static analysis details and dynamic method call graphs. The LLM autonomously determines whether the contextual information suffices to describe the buggy functionality; if not, we supplement it with essential project-specific knowledge from the base. This iterative process unfolds through multi-turn interactions and terminates when the LLM attains adequate confidence to produce a high-quality query. To tackle the second challenge, we implement a multi-granularity code search strategy that conducts retrieval at various program entity levels (i.e., chunk, method, and module), allowing for a more comprehensive depiction of the semantics across different code snippets. Finally, a voting mechanism from a holistic perspective is employed to pinpoint the fault localization results.

We comprehensively evaluated the effectiveness of CosFL on Defects4J [22] and GrowingBugs [19] benchmarks, which consist of 835 real bugs from 23 Java projects. The results indicate that CosFL performs well by successfully pinpointing the fault location within the Top-1 position for 38.8% (324 out of 835) of the bugs. Compared to other code navigation-based approaches, CosFL achieves a significantly higher localization accuracy, surpassing AgentFL [39] and AutoFL [23] on the Top-1 metric by 26.6% and 57.3%, respectively. We further validated the importance of different components in CosFL with an ablation study. Additionally, we conducted a sensitivity analysis to illustrate the impact of different parameter settings on CosFL. The results demonstrate that CosFL maintains robustness across various LLMs and embedding models. Finally, we presented specific case studies to demonstrate how CosFL performs better than other approaches in FL tasks.

Overall, the main contributions of this paper include:

- **Perspective.** To boost the performance of FL, we propose to conduct this task from a new perspective, i.e., semantic code search.
- **Methodology.** Based on our idea, we propose CosFL, a semantic code search-based fault localizer supported by LLMs' strong code understanding and reasoning capabilities. CosFL is publicly available at <https://github.com/IntHelloWorld/CosFL>
- **Experiment.** We comprehensively evaluate CosFL on 835 bugs from 23 projects, the results demonstrate that CosFL significantly outperforms existing LLM-empowered FL techniques.

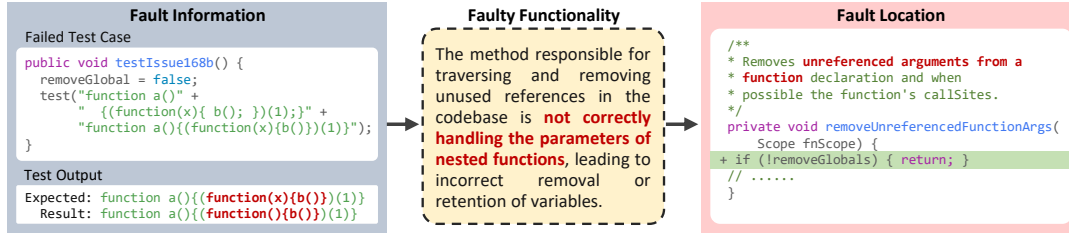


Fig. 2. Concept diagram of localizing Closure-1 bug from the perspective of semantic code search.

2 MOTIVATION

Figure 2 illustrates a bug from the closure-compiler¹ project. The root cause of this bug is that the function arguments were optimized away under unexpected situations. In the *Failed Test Case* testIssue168b, the expected program behavior is to parse the nested function `(function(x)b();)` as `(function(x)b())`. However, in the *Test Output*, the argument `x` has been accidentally removed. To fix this bug, the developers added control logic to the `removeUnreferencedFunctionArgs` method, preventing the program from deleting arguments when the variable `removeGlobal` is set to false. As stated in the code comment, the changed method is responsible for “*Removing unreferenced arguments from a function declaration and when possible the function’s call sites*”.

From the perspective of semantic code search, the new FL paradigm envisioned in this work can be broadly divided into two steps: First, given the *Fault Information* in Figure 2, we comprehensively analyze the root cause of the fault to generate the *Faulty Functionality*, which describes the possible root cause in natural language as “*not correctly handling the parameters of nested function*”. Such a description can indicate the basic semantic information of the buggy function (e.g., we can infer that the buggy function handles the parameters of nested function in this case). Subsequently, we transform the process of searching for suspicious program elements in FL into a CS task, which takes the *Faulty Functionality* as a query to match program elements that are semantically similar to it within the entire codebase. In this case, the approach we proposed successfully identifies the buggy method as the most suspicious location. Intuitively, enhancing FL with CS offers numerous advantages over traditional code navigation-based FL. Code search reviews the entire codebase directly, rather than progressively narrowing the search scope from coarse to fine-grained, which may miss the fault location due to inaccurate decisions. To help further understand the superiority of the new pipeline, we have demonstrated more case studies in Sec 6.1.

However, despite the extensive research on CS and FL over the past decades, few studies have applied the spirit of CS to FL. We attribute this to the absence of prerequisite technologies. Specifically, for semantic-based FL, a significant gap exists between fault-inducing phenomena (such as program output and error stack trace) and fault functionality descriptions that can be used for semantic code search. Traditional statistical learning methods and representation learning approaches struggle to bridge this gap. Fortunately, with the continuous advancement of artificial intelligence technologies, the emergence of large language models has made it possible to automatically diagnose problems in programs [4, 61], thus filling the gap in FL from the CS perspective. In the next section, we will explain our proposed solution in detail.

3 APPROACH

In this work, we propose CosFL, a new paradigm for **F**ault **L**ocalization from the perspective of semantic **C**ode **S**earch. CosFL primarily focuses on method-level FL, since this granularity level is most preferred by developers [25] and also serves as the foundation for many automated software development techniques [63].

¹<https://storage.googleapis.com/google-code-archive/v2/code.google.com/closure-compiler/issues/issue-253.json>

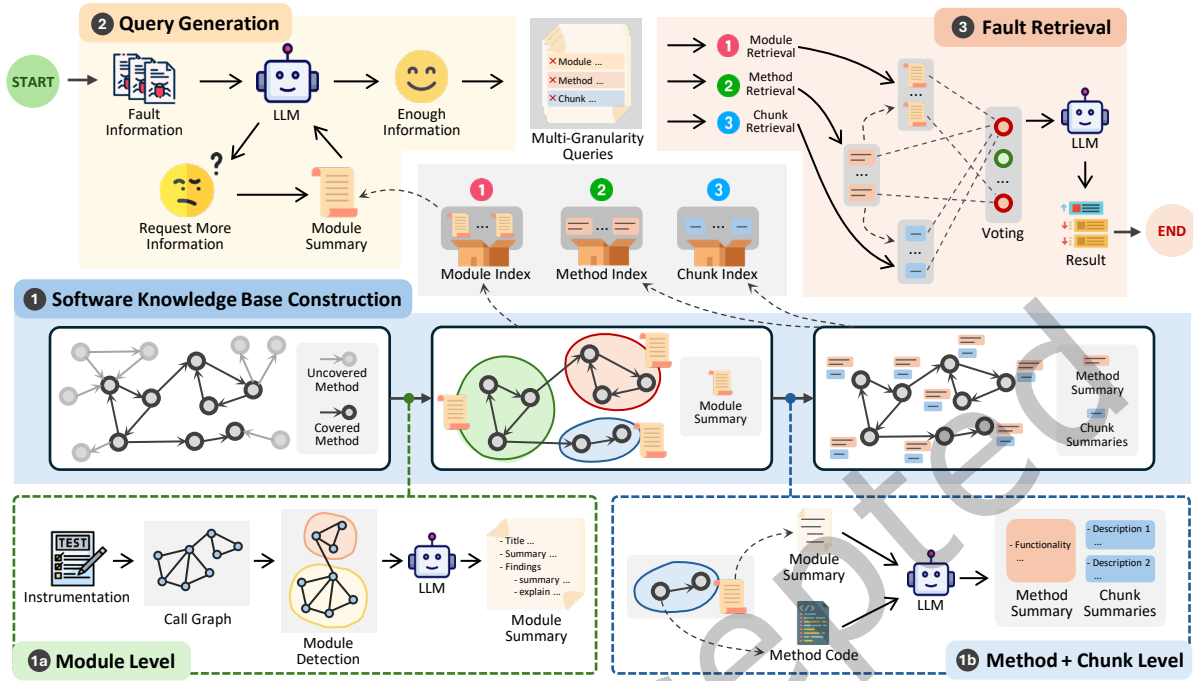


Fig. 3. Overview of CosFL. ① : Construct the Software Knowledge Base based on the method call graph covered by failed tests, which includes two steps 1a Module Level and 1b Method+Chunk Level knowledge extraction. ② : Given the fault information of the failed tests, the LLM requests information from the knowledge base to generate multi-granularity queries that describe the suspicious software functionalities. ③ : Perform multi-granularity code search and finally output the suspicious methods through a voting mechanism.

To achieve better compatibility between CS and FL, CosFL provides two solutions: 1) To generate high-quality queries, CosFL constructs a software knowledge base, providing useful context for LLM to understand the root causes comprehensively. 2) To improve the search precision, we implement multi-granularity semantic CS to more accurately localize buggy methods, and further optimize the localization results through a voting mechanism.

An overview of CosFL is shown in Figure 3. In this work, we define a *module* [27] as a group of closely functionally related methods and a *chunk* [45] as a sequence of continuous statements in a method. These two levels of granularity, describing program semantics from both a holistic and a detailed perspective, respectively, contribute to comprehensively depicting the context of methods and precisely pinpointing the fault locations. In the ① **Software Knowledge Base Construction** phase, we utilize program instrumentation to collect the runtime behavior of the software, and then extract software knowledge at different granularities (i.e., module, method, and chunk) through LLM. In the ② **Query Generation** phase, LLM starts with the fault information to generate queries that describe the suspicious software functionalities in multiple granularities. Particularly, the LLM can actively request module-level knowledge from the Module Index to more comprehensively understand the software's internal behavior. In the ③ **Fault Retrieval** phase, CosFL employs CS to search for program elements at different granularities, and finally output a list of suspicious methods through a voting mechanism.

3.1 Software Knowledge Base Construction

Given software with bugs, CosFL first collects the runtime behavior of the software and uses LLM to extract project-specific knowledge at different granularities. The acquired multi-granularity knowledge will provide informational support for the subsequent FL process. Specifically, we construct the software knowledge base for addressing two limitations: 1) From the fault analysis perspective, existing LLM-based FL methods [23] only provide basic fault information as input, such as test case and test output, while lacking a deep understanding of the complete software architecture and runtime behavior. This may threaten LLM's judgment and consequently affect the FL effectiveness; 2) From the code retrieval perspective, semantically similar code snippets may exist at different locations in one project, necessitating fine-grained context information (e.g., chunk-level semantics) for effective differentiation. As shown in the lower part of Figure 3, the knowledge base construction is divided into two stages. At a coarse granularity, CosFL first collects dynamic runtime information of the software and extracts module-level knowledge. Based on the module-level knowledge, CosFL further extracts method- and chunk-level knowledge for each method at a fine granularity. In the following content, we will introduce these two stages in detail.

Module-Level Knowledge Extraction. At this stage, our objective is to acquire module-level knowledge within the software. According to the consensus in software architecture [27], we naturally consider methods as minimal functional units and define a module as a group of closely functionally related methods. Specifically, to more accurately represent the software behavior that triggers the bug, the functional modules mentioned in this paper are dynamically constructed, that is, all methods comprising a module must have been executed at runtime.

Call Graph Construction. We employ program instrumentation techniques to collect method invocation logs during software execution and generate a dynamic method call graph on the fly. Specifically, we first execute *each failed test case individually* and use instrumentation techniques to track method invocation information, thereby generating a method call graph for each test case. Compared to spectrum-based fault localization techniques [40], CosFL does not require a large-scale test suite that includes both passed and failed test cases, thus significantly reduces execution costs. In each graph, we record the method ID, invocation edges, and the frequency of each invocation. During this process, we exclude methods that are not covered during runtime, which avoid the extraction of knowledge contaminated by untriggered program functionalities. In addition, since we only focus on methods within the target software, utility methods from external libraries (such as `StringUtils.isEmpty()`) will also be excluded in the call graph. Then, we merge all the call graphs into a single global call graph. Ideally, the global call graph contains the covered methods and the invocation relationships between methods during the execution of all failed test cases, which can be used to prompt the LLM with additional internal software information. It is worth noting that we do not maintain an individual call graph for each failed test case. Since different failed test cases triggered by the same bug often cover overlapping methods, maintaining a global call graph is more cost-effective and easier to manage.

Module Detection. However, in large-scale software systems, the size of the dynamic method call graphs can be extremely large. For instance, the call graph of the example mentioned in Figure 2 contains 1,082 nodes and 2,064 edges. Given that LLMs struggle to comprehend inputs of such magnitude [32], it is necessary to appropriately decompose the entire call graph (corresponding to the overall software behavior) into smaller subgraphs (corresponding to the functional modules). To this end, we noticed that the principle of defining functional modules in software engineering is high cohesion and low coupling [15], a characteristic highly similar to communities in networks. In graph theory, a community can be defined as a subset of nodes that are densely connected to each other but sparsely connected to nodes in other communities within the same graph [6]. Inspired by this concept, we identify functional modules within the call graph through the Leiden [50] community detection algorithm. The Leiden algorithm optimizes modularity and identifies tightly connected groups within the network through an iterative process of moving nodes to neighboring communities, refining

the community structure, and then aggregating the communities into new nodes. In the algorithm, the quality function serves as the optimization objective, guiding node movement and community partitioning, evaluating the quality of divisions, and affecting the convergence and outcomes of the algorithm. In this work, we measure the quality of a detected module by the density and frequency of method invocations within it. Our intuition is that the methods within the same module usually collaborate to complete certain tasks, thus tend to more frequently call each other during runtime. Assuming that edge (i, j) in the call graph $G = (V, E)$ have weights w_{ij} (i.e., method i invoke method j for w_{ij} times), the quality function Q of graph G is as follows:

$$Q = \frac{1}{W} \sum_{i,j} \left[w_{ij} - \frac{k_i k_j}{W} \right] \delta(c_i, c_j) \quad (1)$$

Where $W = \sum_{i,j} w_{ij}$ is the sum of all edge weights in the graph, $k_i = \sum_j w_{ij}$ is the weighted degree of vertex i , c_i is the module to which vertex i belongs. $\delta(c_i, c_j)$ is an indicator function whose value equals to 1 if $c_i = c_j$, and 0 otherwise. The value of quality Q ranges between $[-1, 1]$, a higher Q indicates a better partition, implying that the connections within the detected functional modules are tighter, while the connections between the modules are looser. Based on the definition of Q , the Leiden algorithm is able to continuously search the optimal partition of the call graph. Specifically, we set the initial partition of the algorithm as that each node is treated as an individual community. The termination condition is defined as either the quality metric Q no longer improving or the process reaching the maximum number of iterations, which is set to 5. We then regard each clustered call subgraph as a candidate function module.

Model Size Optimization. For module detection, our objective is to identify functional modules that encapsulate module-level knowledge and can be comprehensively analyzed by the LLMs. However, the clustering process of the Leiden algorithm is greedy, which may produce functional modules that have very different sizes from each other. As shown in Figure 4a, under the default Leiden algorithm, the sizes of the clustered modules approximate an exponential distribution. We observed that there are 1,623 small modules with sizes in the range of $[0, 5)$, accounting for 21.6%. Meanwhile, large modules with sizes greater than 60 constitute 13.2% of the total cases. To ensure the validity of CosFL, we argue that it is crucial to regulate the size of each module, keeping it within an appropriate and balanced range. This regulation serves two key purposes: First, modules that are too small should be avoided because they typically involve interactions among only a few methods. Such a limited scope fails to capture broader architectural patterns, resulting in a lack of holistic software structure information. Second, excessively large modules should also be prevented, as they encompass too many methods. When a module becomes too large, the overwhelming context would make it challenging for the LLM to effectively analyze and summarize the core insights [32]. Therefore, striking the right balance in module size is essential, as it ensures that each module is neither too narrow to miss important architectural context nor too broad to hinder meaningful summarization by LLMs. This balance allows CosFL to maintain both precision and comprehensiveness in its analysis.

To address the above issue, we propose to optimize the model size through two heuristic strategies. First, the Leiden algorithm supports setting a parameter to limit the maximum module size. This configuration prevents the algorithm from clustering excessively large modules, but may still result in overly small modules. Next, we further designed an algorithm to prevent the formation of excessively small modules. As shown in Algorithm 1, the input is all of the detected modules C and a parameter N to regulate the minimal module size. For each undersized module c'_i (line 2), we first select the most closely connected adjacent module c_t as the target module (lines 4-16), which is determined by the magnitude of the total edge weights between modules (lines 7-13). Subsequently, we migrate all method nodes from the undersized modules to the target module c_t (lines 17-19), thus eliminating small modules without losing any node.

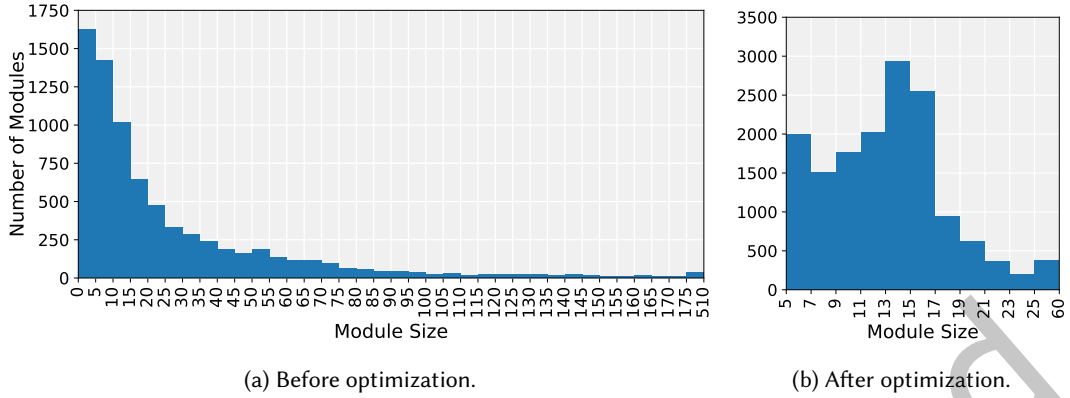


Fig. 4. Module size distribution before and after optimization. We place the bars between two scales (e.g., 5 and 10) in the figure to indicate the number of modules whose sizes are within this interval (e.g., $[5, 10)$).

Algorithm 1: Eliminate undersized modules.

Input: N : the minimal module size. C : All modules.

Output: C : The changed modules.

```

1   $C' = \text{getUndersizedModules}(C, N)$  ; /* Get modules in  $C$  whose size is smaller than  $N$  */
2  for  $c'_i$  in  $C'$  do
3       $V = \text{getNodes}(C')$  ; /* All method nodes in  $c'_i$  */
4       $c_t = \text{None}$  ; /* The target appropriate-size module for  $c'_i$  */
5       $\epsilon = 0$  ; /* A variable to record the weight between the undersized and target module */
6      for  $c_i$  in  $C - C'$  do /* Record the weight between two modules */
7           $\epsilon' = 0$  ;
8          for  $v_i$  in  $V$  do /* Get all modules that have calling relationships with method  $v_i$  */
9               $C^* = \text{getNeighbors}(v_i)$  ;
10             for  $C_i^*$  in  $C^*$  do
11                 if  $C_i^* == c_i$  then /*  $w$  is the sum of edge weights between node  $v_i$  and module  $C_i^*$  */
12                      $w = \text{getWeight}(v_i, C_i^*)$  ;
13                      $\epsilon' += w$ 
14             if  $\epsilon' > \epsilon$  then
15                  $\epsilon = \epsilon'$  ;
16                  $c_t = c_i$ 
17 for  $v_i$  in  $V$  do
18      $c_t.\text{addNode}(v_i)$  ;
19      $c'_i.\text{removeNode}(v_i)$  ;
20 Return  $C$ 

```

To determine the appropriate range for module size, we predefined three range settings (i.e., [0, 10), [5, 15), and [10, 20)) and evaluated the FL performance of CosFL under varying module size ranges. Specifically, we selected 75 bugs from three projects within the Growing-Bugs [19] dataset as the validation set, including 27 bugs from project *commons-beanutils*, 18 bugs from project *commons-dbc*, and 30 bugs from project *Scxml*. As illustrated in Figure 5, the results reveal that CosFL achieves the best performance under module size range [5, 15), it successfully locates 35, 52, and 55 bugs within Top-1, Top-3, and Top-5, respectively. Consequently, for the default version of CosFL, we configured the minimum module size parameter to 5 and the maximum module size parameter to 15. The distribution of the model sizes after optimization is shown in Figure 4b, where 97.5% of the modules fall within the size range of [5, 25). Note that since we eliminate overly small modules by merging them into nearby modules, the existence of modules with a size greater than 15 is normal.

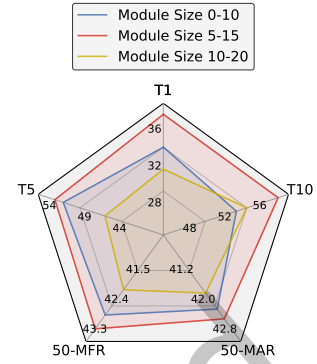


Fig. 5. The performance of CosFL under different module size settings.

Prompt 1: Module-Level Knowledge Extraction

You are an expert software analyst with deep knowledge of program structure.

Goal

As a senior software engineer specializing in code analysis, generate a comprehensive functionality summary report for a method call subgraph from a program execution. This report should include an overview of the key methods in the subgraph and their calling relationships.

Call Graph

Methods: <Information of the method nodes in the call graph>

Calls: <Call relationships between method nodes>

Report Structure

The report should include the following sections:

- TITLE: A short but specific title representing the main functionality of the call graph ...
- SUMMARY: An executive summary of the subgraph's overall structure ...
- DETAILED FINDINGS: A list of insights about the call graph ...

Example

Methods:

id,className:methodName(startLine-endLine),comment
1,Main:main(1-10),The main entry point of the program

...

Calls:

id,source,target
1,Main:main(1-10),DataProcessor:processData(11-20)

...

Output:

"title": "Data Processing and Validation Flow"

"summary": "This subgraph demonstrates a simple data processing and validation flow ..."

"findings":

"summary": "Central role of processData method"

"explanation": "The processData method is not only responsible for data processing but also ensures data validity by calling ..."

...

Knowledge Extraction. After module detection, we match each functional module in the dynamic call graph with the static code repository, which provides additional attributes for each method node in the graph, such as method comments, source code, etc. Finally, we leverage the graph structure data analysis capabilities [8, 12] of LLMs to extract knowledge at the functional module level, which is expressed in the form of natural language module summary as shown in Figure 3. An example prompt for the LLM is shown in Prompt 1. For each module (represented by a call subgraph) to be summarized, we transform the method nodes and call edges in the graph into the text format that can be interpreted by LLMs. Acting as an *expert software analyst*, the LLM is instructed to “generate a comprehensive functionality summary report” for each functional module. The generated report in # *Report Structure* should consists of three sections: *TITLE* names the overall functionality of the module; *SUMMARY* summarizes the overall structure of the module and how methods interact with each other; and *DETAILED FINDINGS* is a list of insights that elaborate on the behavior of some important methods.

Prompt 2: Method- & Chunk-Level Knowledge Extraction

You are a code summarizer with deep knowledge of different programming languages.

Goal

As a senior software engineer specializing in code summarization, your task is to generate comprehensive documentation for a given method. This documentation should succinctly describe the key functionality of the method and provide a detailed walkthrough of its workflow.

Provided Information

Method Code: <The method code to be summarized>

Developer Comment: <Any additional comments or insights provided by the developer, if available>

Module Context: <The broader context in which the method operates, if available>

Report Structure

The report should include the following sections:

- FUNCTIONALITY: A summary of the method, its role within the broader module context ...
- DESCRIPTION: A list of detailed step-by-step explanation of the method's workflow.

Example

Method Code:

```
public void processTasks(List<Task> tasks) {
    while (!tasks.isEmpty()) {
        Task task = getNextTask(tasks);
        handleTask(task);
    }
}
```

Developer Comment:

Handle a list of tasks by some order. Return when all tasks are done.

Module Context:

"title": "Data Processing and Validation Flow"

"summary": "This subgraph demonstrates a simple data processing and validation flow ..."

...

Output:

"functionality": "Manages the sequential processing of tasks in a list until all are completed."

"description":

"Initially checks if the task list is not empty. If tasks remain, it retrieves the next task ..."

"Each retrieved task is then processed by the handleTask method"

...

Method- & Chunk-Level Knowledge Extraction. In this stage, our objective is to acquire knowledge at the method and chunk levels within the software. We observe that existing LLM-based FL techniques typically require method-level semantic information to search for suspicious locations. However, the semantic information maintained by these techniques for individual methods is limited to the internal scope of the method, without considering the runtime context in which the method is executed. For instance, AutoFL [23] directly utilizes the

method code and comments as information sources, AgentFL [39] further employs LLMs to enhance method comments, but the semantic information remains confined within a few methods. To address this, we propose to consider the runtime context of the method when extracting method-level knowledge. For each method, its context is represented by the functional module summary (we have obtained in the previous stage) where the method is located.

Nevertheless, due to the characteristics of object-oriented languages [26] and the prevalence of code clone [41], many methods in code base may exhibit similarity in overall functionality while differing in internal logical details. In such situations, CS based solely on method-level semantics may struggle to distinguish subtle differences between similar methods, thereby compromising the FL performance. Consequently, we utilized LLM to summarize the detailed workflow of each method, thus decomposing the overall functionality of a method into finer-grained chunk-level units. Here, we define a chunk as a program logic unit composed of one or more continuous statements in a method. With the chunk-level knowledge, CosFL can differentiate similar methods through exploiting fine-grained variances in their detailed implementations, thereby further improving the FL accuracy.

Prompt 3: Query Generation

You are a Software Diagnostics Specialist specializing in software fault analysis and fault localization.

Goal

Your task is to analyze the provided fault information and either identify the potentially faulty functionality or request additional information as needed.

Fault Information

Test Case Code: <Source Code of the Failed Test Case>

Exception Stack Trace: <Exception Stack Trace>

Test Output: <Test Output>

Module Details: <The Retrieved Module Summaries>

Analysis Process

1. Analyze provided information and interactions between methods that might cause the fault.
2. Formulate a specific question to gather necessary details.
2. Describe the potentially faulty functionality at various levels of detail.

Response Format

- Specifying the additional information needed to complete the analysis, the response should contain a 'request' field:
 - "request": "<Module details or specific information you needed to better understand the fault>"
- Describing the likely location and nature of the bug at increasing levels of detail, the response should contain 'module', 'method', and 'chunk' fields:
 - "module": "<A description of the likely functional module where the bug resides>"
 - "method": "<A description of the functionality in the software which may cause the fault>"
 - "chunk": "<A more detailed description of the specific code logic that is likely causing the bug>"

An example prompt is shown in Prompt 2, for each method, we generate both method- and chunk-level knowledge in one round of conversation with the LLM. In # *Example*, it can be observed that the semantic information for each method is derived not only from the *Method Code* and *Developer Comment*, but also includes the corresponding *Module Context*. As an expert *code summarizer*, the LLM is tasked to “describe the key functionality and provide a detailed walkthrough of the workflow” for each method. The generated report in # *Report Structure* has two parts, where *FUNCTIONALITY* provides a detailed description of the method’s functionality and its role within the module context, and *DESCRIPTION* is a list of paragraphs that explain the method’s workflow in detail, which is used as the chunk-level knowledge of this method.

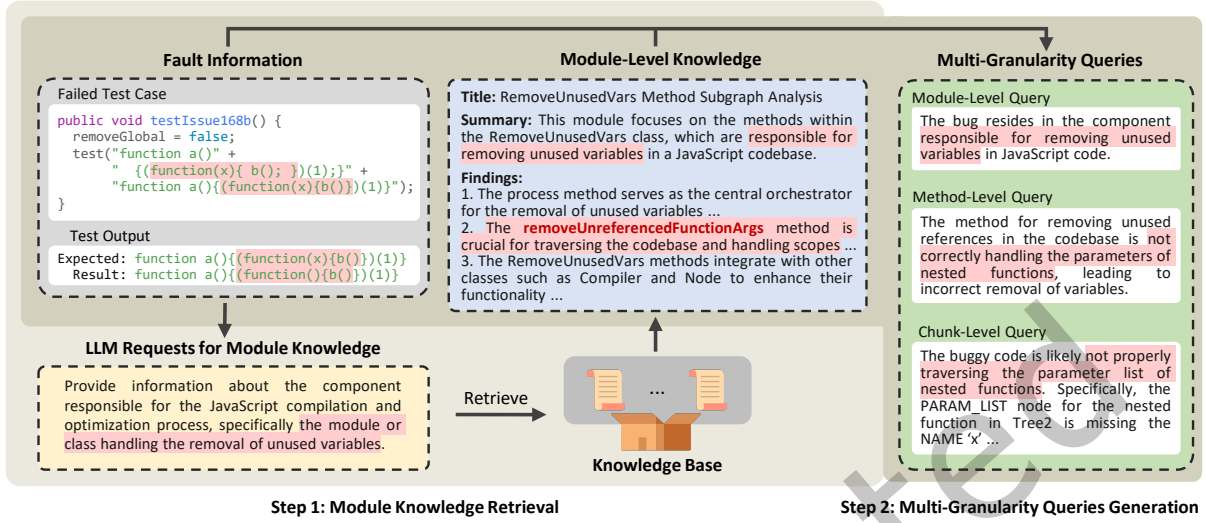


Fig. 6. A running example for query generation. We emphasize the crucial “hints” for LLM in red.

3.2 Query Generation

As shown in Figure 3, the objective of the *Query Generation* phase is to provide the *Multi-Granularity Queries* as the input for the following *Fault Retrieval* process. Specifically, to more comprehensively depict the characteristics and context of suspicious software functionalities, CosFL generates three queries within different granularities (i.e., module, method, and chunk). However, reconsidering the example in Figure 2, the fault information typically only contains the software’s expected behavior (Failed Test Case), actual behavior (Test Output, which is sometimes not available), and a small portion of the execution path (such as the exception stack trace), while the vast majority of the software’s internal logic remains a black box. This prevents LLM from comprehensively understanding the root cause, let alone generating high-quality queries. To mitigate this problem, we allow the LLM to retrieve the relevant functional module knowledge from the constructed *Software Knowledge Base* before it generates the multi-granularity queries. The LLM prompt is shown in Prompt 3, we use the italic text and dashed line to mark the prompt for the two steps.

Step 1: Module Knowledge Retrieval. For each failed test, we first instruct the LLM to analyze the basic fault information and “*formulate a specific question to gather necessary details*”. As described in # *Response Format*, the LLM is required to respond with a *request* which describes the needed functional module details. In this step, we use semantic search to find the most relevant module summary from the module index and add it to the *Module Details* section within # *Fault Information*. It is worth noting that we restrict the LLM to retrieve module-level knowledge. This approach facilitates the LLM’s global understanding of potential root causes of the bug while preventing it from generating overly specific queries that lack generalizability.

Step 2: Multi-Granularity Queries Generation. After being provided with the module knowledge that is relevant to the buggy software functionality, the LLM is then asked to “*Describe the potentially faulty functionality at various levels of detail*”. Specifically, the LLM is asked to generate three fields (e.g., *module*, *method*, and *chunk*) to depict the suspicious functionality in the software at different levels of granularity. These fields will be used as queries for the subsequent multi-granularity code search process.

Running Example. A running example for the same bug we have mentioned in Section 2 is illustrated in Figure 6. In *Step 1: Module Knowledge Retrieval*, the LLM initially assesses the potential cause of the bug from the

fault information and generates a request to gain further insight into “the module or class handling the removal of unused variables”. Subsequently, CosFL utilizes this request to retrieve the most relevant module information from the knowledge base. The *Summary* field indicates that this module mainly “focuses on the methods within the *RemoveUnusedVars* class, which are responsible for removing unused variables”. Specifically, the functionality of the buggy method *removeUnreferencedFunctionArgs* is also included in the *Findings* field. For *Step 2: Multi-Granularity Queries Generation*, CosFL incorporates both fault information and retrieved module-level knowledge to further analyze the root cause and generate multi-granularity queries. As shown in the figure, the generated queries exhibit high semantic relevance to the buggy method. For instance, the Method-Level Query suggests the fault might stem from a method that “not correctly handling the parameters of nested functions”. With these multi-granularity queries, the subsequent code search process is able to pinpoint the bug-related methods from the codebase.

3.3 Fault Retrieval

As illustrated in the upper right of Figure 3, the *Fault Retrieval* phase takes the *Multi-Granularity Queries* as input, which aims to pinpoint the suspicious buggy methods from the perspective of semantic code search (CS). To achieve this, we leverage the text embedding [16] technique which is widely used in CS [14]. Text embedding models map natural/program language text into a high-dimensional vector space, where semantically similar texts are positioned closer to each other. For the fault localization task, our insight is that the suspicious functionalities derived from LLM analysis and the actual buggy location can be semantically similar. Specifically, the workflow of *Fault Retrieval* consists of two steps:

Multi-Granularity Retrieval. We first build embedding indexes for program elements at different levels using a text embedding model, where the embedded textual content is derived from the knowledge acquired in the previous Section 3.1. Formally, we represent all program elements as $E = \{G, M, S\}$, where $G = \{g_1, \dots, g_{|G|}\}$ denotes function modules (i.e., dynamic call subgraphs), $M = \{m_1, \dots, m_{|M|}\}$ represents methods, and $S = \{S^{m_1}, \dots, S^{m_{|M|}}\}$ comprises all chunks for all methods, with $S^{m_i} = \{s_1^{m_i}, \dots, s_{|S^{m_i}|}^{m_i}\}$ being the chunks in method m_i . We then utilize a text embedding model σ to vectorize the knowledge text, which results in the *Module Index* $I^G = \{\sigma(\kappa(g_1)), \dots, \sigma(\kappa(g_{|G|}))\}$, *Method Index* $I^M = \{\sigma(\kappa(m_1)), \dots, \sigma(\kappa(m_{|M|}))\}$, and *Chunk Index* $I^S = \{\sigma(\kappa(S^{m_1})), \dots, \sigma(\kappa(S^{m_{|M|}}))\}$ as shown in Figure 3. Here, κ maps each program element to its corresponding knowledge summary extracted by the LLM.

Next, let $U = \{u_i = (c_i, f_i, l_i) | i = 1, 2, \dots, N\}$ denote the multi-granularity queries generated in the *Query Generation* phase, where c , f , and l are queries respectively corresponding to module-, method-, and chunk-level suspicious functionalities, N is the number of failed test cases. The knowledge base actually maintains three granularities of knowledge $k_m = (\kappa(\phi(m)), \kappa(m), \kappa(\varphi(m)))$ for each method m , where $\phi : M \mapsto G$ maps a method to the module in which it is located, and $\varphi : M \mapsto S$ maps each method m to the set of chunks S^{m_i} within that method. Given the above condition, the retrieval process can be formally described as:

$$\hat{M}_i = \{\hat{m}_1, \dots, \hat{m}_{|\hat{M}_i|}\} = \text{Retrieve}(I^M, f_i, \lambda) \quad (2)$$

$$\hat{G}_i = \{\hat{g}_1, \dots, \hat{g}_{|\hat{G}_i|}\} = \text{Retrieve}(I_p^G, c_i, \lambda), \quad I_p^G = \{\phi(\hat{m}_1), \dots, \phi(\hat{m}_{|\hat{M}_i|})\} \quad (3)$$

$$\hat{S}_i = \{\hat{s}_1, \dots, \hat{s}_{|\hat{S}_i|}\} = \text{Retrieve}(I_p^S, l_i, \lambda), \quad I_p^S = \varphi(\hat{m}_1) \cup \dots \cup \varphi(\hat{m}_{|\hat{M}_i|}) \quad (4)$$

$$\hat{E} = \{(\hat{G}_i, \hat{M}_i, \hat{S}_i) | i = 1, \dots, N\} \quad (5)$$

The Retrieve function measures the semantic similarity between the query and program element summaries using cosine similarity [48], and returns the suspicious program elements in descending order of similarity. Here, λ is a hyper-parameter used to control the number of retrieved program elements. Specifically, for each failed test case, Formula 2 is used to retrieve suspicious methods, while Formulas 3 and 4 are used to retrieve suspicious

modules and chunks, respectively. Formula 5 denotes the results from all failed test cases. Note that we use the results from method retrieval $\hat{M}_i$ to narrow down the size of module and chunk index (I_p^G and I_p^S), which improves retrieval efficiency by excluding a large number of irrelevant program elements.

Suspicious Method Voting. After obtaining all retrieved program elements $\hat{E}$ in formula 5, we are supposed to aggregate the retrieval results from multiple queries and granularities, which finally results in a list of methods sorted in descending order of suspiciousness. Inspired by the Borda count [13] in social science, we designed a voting mechanism to elect the more suspicious methods. Our insight is that if a method (including its related module and chunks) is semantically related to more queries, then this method is more likely to be the actual fault location. Formally, for the set of all retrieved methods $M^* = \hat{M}_1 \cup \dots \cup \hat{M}_N$, the suspicious score of an individual method $m^* \in M^*$ is calculated using the following formula:

$$\text{Score}(m^*) = \sum_{i=1}^N \sum_{j=1}^{|\hat{G}_i|} \omega(\hat{g}_{ij}) + \sum_{i=1}^N \sum_{j=1}^{|\hat{M}_i|} \omega(\hat{m}_{ij}) + \sum_{i=1}^N \sum_{j=1}^{|\hat{S}_i|} \omega(\hat{s}_{ij}) \quad (6)$$

$$\omega(\hat{g}_{ij}) = \begin{cases} \epsilon(\hat{g}_{ij}), & \text{method } m^* \text{ belongs to module } \hat{g}_{ij} \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

$$\omega(\hat{m}_{ij}) = \begin{cases} \epsilon(\hat{m}_{ij}), & m^* = \hat{m}_{ij} \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

$$\omega(\hat{s}_{ij}) = \begin{cases} \epsilon(\hat{s}_{ij}), & \text{chunk } \hat{s}_{ij} \text{ belongs to method } m^* \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

ϵ is computed during the retrieval process with a value range of $[-1, 1]$, representing the semantic similarity between a program element and its corresponding query. As shown in Formula 6, the suspiciousness of a method m^* not only stems from its similarity to method-level queries (calculated by the second term of the formula), but also from the degree to which the associated module and chunks of the method match to the corresponding queries (the first and third terms). After obtaining the suspiciousness score for each method m^* , all methods in M^* will be ranked in descending order. It is worth noting that due to the combination of the multi-granularity retrieval mechanism and the cosine similarity-based suspiciousness calculation, we did not find the phenomenon of ties in suspicious scores in our experiments, thus avoiding additional designs for breaking ties in potential faulty methods.

Finally, we follow previous studies [23, 25] to generate an explanations for each localized suspicious method. Specifically, an LLM is employed to generate an bug explanation by summarizing the fault information and all products of the CosFL (including the multi-granularity queries and the source code of the retrieved suspicious method). Although this operation does not directly affect the FL performance of CosFL, the bug explanations generated by the LLM can help users better evaluate the authenticity of CosFL results.

4 EXPERIMENT

4.1 Research Question

To comprehensively evaluate the performance of CosFL, we design three research questions:

RQ1: The effectiveness of CosFL. How effective is CosFL in method-level fault localization compared to other state-of-the-art code navigation-based FL techniques?

RQ2: The impact of different components in CosFL. We conduct an ablation study to investigate how different components contribute to the overall performance of CosFL.

Table 1. Benchmark Information.

	Project	Name	#Bug	LoC	NoM
Defects4J V1.2.0	Chart	jfreechart	25	207K	14,925
	Closure	closure-compiler	125	123K	18,052
	Lang	commons-lang	61	56K	8,325
	Math	commons-math	99	164K	20,503
	Mockito	mockito	35	20K	2,522
	Time	joda-time	25	61K	3,993
Defects4J V2.0.0	Closure	closure-compiler	41	123K	18,052
	Cli	commons-cli	37	4K	1,400
	Codec	commons-codec	15	5K	1,655
	Collections	commons-collections	3	61K	5,004
	Compress	commons-compress	44	30K	5,666
	Csv	commons-csv	15	2K	592
	Gson	gson	14	10K	1,271
	JacksonCore	jackson-core	19	24K	3,889
	JacksonDatabind	jackson-databind	39	65K	13,861
	JacksonXml	jackson-dataformat-xml	6	6K	523
	Jsoup	jsoup	88	4K	2,964
GrowingBugs	IO	commons-io	22	11K	3,646
	Validator	commons-validator	11	16K	1,292
	Javapoet	javapoet	16	1K	139
	Zip4j	zip4j	47	7K	1,442
	Spoon	spoon	16	35K	6,067
	Markedj	markedj	13	2K	168
	Dagger_core	dagger-core	19	3K	382
Overall			835	1,066K	139K

RQ3: Effects of different hyper-parameters on CosFL. We explore the sensitivity of CosFL to different parameter settings.

RQ4: How is the quality of the queries generated by CosFL? We comprehensively evaluate the quality of the generated queries through both automated evaluation metrics and manual analysis.

4.2 Benchmark

We design CosFL as an FL approach for localizing general defects within a software repository. Given the test failure information triggered by a bug as input, CosFL can be used to localize different types of bugs (e.g., crash bug, runtime exception, or logic error). In order to comprehensively evaluate the effect of CosFL on different software projects, we adopt a benchmark which consists of 835 real-world bugs from 23 Java projects. Specifically, we reproduce 691 bugs within 16 open-source Java projects from the widely used software defect benchmark Defects4J [22] (of which Defects4J V1.2.0 contains 370 bugs from 6 projects, and Defects4J V2.0.0 adds 321 bugs from 11 projects). Additionally, we collected 144 reproducible bugs of 7 projects from the GrowingBugs [19] benchmark, which has scarcely been utilized for evaluating the existing FL approaches.

Table 1 shows the detailed information of the benchmark. The columns “Project” and “Name” represent the abbreviation and full name of the software project, respectively. The column “#Bug” represents the number of bugs in the project. “LoC” and “NoM” show the total number of lines of code and the number of methods of the software project, respectively, where the number of methods does not include methods without bodies such as abstract methods. Finally, the benchmark we used to conduct the experiments comprises 835 bugs from 23 Java projects.

4.3 Baseline

We compare CosFL with two state-of-the-art fault localization approaches, AutoFL [23] and AgentFL [39]. Both of them focus on localizing the fault location from the entire software project through code navigation.

AutoFL [23] utilizes the function calling capabilities of LLMs for fault localization. Given functions such as `get_class_covered`, `get_method_covered`, and `get_code_snippet`, the LLM is required to autonomously decide whether to invoke functions to acquire additional information in each round of dialogue, and gradually navigating to the buggy location. It is worth noting that AutoFL employs repeated runs to reduce the perplexity of results. For the sake of fairness, we set the number of runs for all FL tools to 1 in this paper.

AgentFL [39] defines the fault localization task as a standard operating procedure, which is decomposed into three steps: fault comprehension, codebase navigation, and fault confirmation. In each step, AgentFL employs agents capable of utilizing various tools to address specific subtasks. AgentFL incorporates a document-guided search strategy for codebase navigation, which utilizes both existing documents and LLM-enhanced documents to identify classes and methods related to the potential root causes.

4.4 Metrics

We use three metrics to evaluate the method-level FL performance:

Top-N Recall (Top-N). For a single bug version $b \in B$, Top-N indicates whether there is a buggy method $m \in M$ in the first N methods of the method ranking list L provided by the fault localization approach. For an entire project, Top-N is equal to the sum of the results of all bug versions. The formula of Top-N is $\text{Top-N} = \sum_{i=1}^{|B|} I_i$, where I_i equals to 1 if $\exists m \in \{L_1, L_2, \dots, L_N\}, m \in M$, or 0 otherwise. In this work, we adopt $N \in \{1, 5, 10\}$. The higher the Top-N recall, the better the fault localization performance.

Mean First Rank (MFR). MFR can be used to indicate the ranking of the first buggy method in the method ranking list. For a single buggy version, MFR is the ranking of the first buggy method in the list. For all buggy versions in the entire project, MFR is equal to the mean of the results of all buggy versions. The formula of MFR is $\text{MFR} = \frac{1}{|B|} \sum_{i=1}^{|B|} \text{rank}_i$. If no buggy method is found within the recall set of size N , the rank value is assigned to $N + 1$. The lower the MFR, the better the fault localization performance.

Mean Average Rank (MAR). MAR is utilized to indicate the ranking of all buggy methods in the method ranking list. For a single buggy version, MFR is the average ranking of all buggy methods in the list. For all buggy versions in the entire project, MFR is equal to the mean of the results of all buggy versions. The formula of MAR is $\text{MAR} = \frac{1}{|B||M|} \sum_{i=1}^{|B|} \sum_{j=1}^{|M|} \text{rank}_{ij}$. The lower the MAR, the better the fault localization performance.

4.5 Implementation

We implement CosFL based on LlamaIndex [33]. We use ChromaDB [9] as the vector store and tree-sitter [51] for AST-based parsing and program element extraction. For inspecting software dynamic behavior, we conduct program instrumentation through a self-implement Java agent [17] and build the dynamic method call graph on the fly with JGraphT [18]. We set the following alterable parameters for the default version of CosFL. In *Module-Level Knowledge Extraction* of Section 3.1, the maximum and minimum module sizes are set to 15 and 5 respectively. The number of retrieved elements in *Fault Retrieval* of Section 3.3 is set to $\lambda = 50$.

We choose jina-embeddings-v2-base-en [21] as the default text embedding model. At the time of writing this paper, the DeepSeek-V2.5 series model had achieved comparable performance to other mainstream general-purpose LLMs in coding abilities. Our sensitivity analysis in Section 8 also confirms this. Additionally, its cost is significantly lower than other LLMs, with a price of only approximately 0.14 (0.29) for one million input (output) tokens, which is less than 10% of the cost of GPT-4o series models. Considering the relatively high computational resources required to adopt the GPT series model for all LLM-based approaches, we choose to employ DeepSeek-V2.5-1210 [10] as the LLM backend for CosFL due to its high cost-effectiveness. For a fair comparison, we also

Table 2. Results of CosFL and other code navigation based FL approaches.

Project	Bugs	AutoFL					AgentFL					CosFL				
		T1	T5	T10	MFR	MAR	T1	T5	T10	MFR	MAR	T1	T5	T10	MFR	MAR
Chart	25	6	9	9	33.16	34.16	12	18	18	15.40	15.74	15	23	23	4.24	5.04
Closure	166	3	4	4	49.79	50.39	27	53	61	24.17	24.50	35	61	73	29.34	30.25
Lang	61	27	31	31	25.67	28.27	38	46	46	13.49	13.61	36	52	55	5.20	5.42
Math	99	43	56	56	22.90	26.04	48	64	64	18.96	19.11	46	74	75	8.80	9.53
Mockito	35	15	16	16	28.17	38.89	16	18	18	25.37	25.59	15	23	24	13.60	14.74
Time	25	10	11	11	29.08	34.25	12	14	14	23.08	23.10	7	13	13	19.16	20.14
Cli	37	6	10	10	37.25	37.25	12	22	22	21.78	21.96	15	26	30	6.24	7.82
Codec	15	4	5	5	33.21	33.60	5	12	12	11.80	11.93	6	13	14	3.00	3.84
Collections	3	0	0	0	50.33	50.33	0	0	0	51.00	51.00	1	1	1	34.33	34.33
Compress	44	16	20	20	28.45	30.71	17	28	28	19.55	19.55	18	30	33	9.16	10.52
Csv	15	4	4	4	37.67	41.00	4	5	5	34.47	34.47	9	11	11	5.07	5.37
Gson	14	4	6	6	29.79	33.87	6	7	7	26.07	26.14	8	11	11	6.64	6.75
JacksonCore	19	3	6	6	35.47	38.00	7	11	11	22.42	22.73	9	15	15	8.05	8.75
Jackson-Databind	39	4	5	5	44.54	50.08	4	6	7	42.31	42.32	5	15	16	28.13	28.55
JacksonXml	6	1	1	1	42.50	42.50	1	1	1	42.67	42.67	3	4	4	16.50	16.50
Jsoup	88	25	25	25	36.75	40.57	21	37	37	30.31	30.34	31	52	56	16.83	17.78
IO	22	12	18	18	10.36	14.15	0	0	0	51.00	51.00	13	21	21	2.18	2.30
Validator	11	2	3	3	34.33	34.33	5	9	9	10.45	10.45	8	10	10	5.91	6.16
Javapoet	16	7	7	7	29.13	30.69	10	14	14	7.63	7.63	12	16	16	1.25	1.66
Zip4j	47	10	10	10	40.36	42.28	6	8	8	42.62	42.65	15	29	30	14.55	16.09
Spoon	16	3	3	4	38.81	40.22	2	7	7	29.56	29.56	6	8	9	19.88	20.63
Markedj	13	0	0	0	51.00	51.00	1	2	2	43.38	43.38	5	11	12	3.31	6.28
Dagger_core	19	1	3	3	43.11	47.40	2	2	2	45.74	45.74	6	13	14	12.79	15.16
Total	835	206	253	254	35.78	38.48	256	384	393	26.80	26.90	324	532	566	14.15	14.97

use the same model to reproduce the state-of-the-art approaches AutoFL [23] and AgentFL [39]. To ensure the scalability of CosFL, we further investigated its FL performance across other mainstream models in Section 5.3.

5 EVALUATION

5.1 RQ1: Effectiveness of CosFL

In Table 2, CosFL is compared with other LLM-based techniques on method-level FL effectiveness. Overall, CosFL achieves the best performance, which localizes 324, 532, and 566 bugs in the top 1, 5, and 10 positions, respectively. The MFR is 14.15 and the MAR is 14.97, indicating that CosFL can rank the buggy methods at relatively high positions. The effect of AgentFL is inferior to CosFL, which localizes 256, 384, and 393 methods in the top 1, 5, and 10 positions, with an MFR of 26.80 and MAR of 26.90. AutoFL performed the worst among the three approaches, with Top1, Top3, and Top5 scores of 206, 253, and 254, respectively, and MFR of 35.78 and 38.48. CosFL shows 57.3% and 26.6% increases in Top1 compared to AutoFL and AgentFL, respectively. For the Top5 accuracy, the increases are even more significant, reaching 110.3% and 38.5%. Such results are consistent with our expectation that the performance of FL techniques is proportional to the amount of software information they can exploit. Specifically, AutoFL only utilized the most basic information such as the names of the covered classes and methods; AgentFL further incorporates documentation generated by developers or LLMs, providing more detailed knowledge about software functionality; CosFL provides the richest information through a software knowledge base, which supplies runtime information about bugs, offering a basis for more accurate fault localization.

In Figure 7, we present the overlap of Top5 results among different FL approaches. Particularly, we employed the Top5 metric to measure the comprehensive localization performance, as a previous study [25] indicated that over 70% of practitioners consider localization results that rank buggy elements within the top five positions to be successful. As shown in the figure, we observe that the results of CosFL substantially cover the other two, identifying 80.7% (310 out of 384) of the bugs localized by AgentFL and 87.7% (222 out of 253) of AutoFL. Moreover, CosFL independently finds 176 bugs, which illustrates the effectiveness of boosting FL through semantic code search.

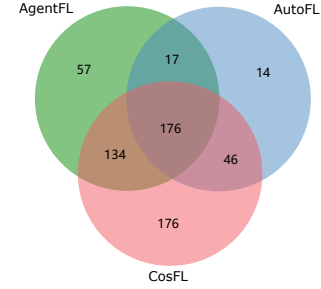


Fig. 7. Overlap of Top5 results.

Answer to RQ1: CosFL shows superior performance in fault localization by ranking 324 (532) out of 835 bugs within the top 1 (5) positions, significantly outperforming other LLM-based methods.

5.2 RQ2: Ablation Study

To evaluate the contribution of different components in CosFL, we designed three variants:

- **w/o Module Context:** This variant removes the ①a *Module Level* knowledge extraction phase in ① *Software Knowledge Base Construction* phase of Figure 3, thereby impacting both the ② *Query Generation* and ③ *Fault Retrieval* phases: (1) For the *Query Generation* phase, this modification deprives the LLM of access to internal software information via the knowledge base, making it degrade to generate queries based on merely the raw fault information (e.g., test output and error stack trace); (2) This variant also indirectly impacts the accuracy of the method and chunk retrieval process within the *Fault Retrieval* phase. Specifically, in the ①b *Method+Chunk Level* knowledge extraction phase, the absence of the *Module Summary* further prevents the LLM from obtaining relevant functional module information when extracting knowledge at the method and chunk levels.
- **w/o Module Retrieval:** This variant eliminates module-level retrieval in the ③ *Fault Retrieval* phase, which aims to investigate the role of module-level knowledge in retrieving buggy methods.
- **w/o Chunk Retrieval:** This variant excludes chunk-level retrieval in the ③ *Fault Retrieval* phase, which aims to investigate the role of finer-grained chunk-level knowledge in retrieving buggy methods.

The effectiveness of each variant is presented in Table 3. In the first row of the table, we observe that module-level knowledge has a significant impact on CosFL. Interestingly, after removing the module context, the Top1 value drops sharply by 80, while the Top5 remains relatively unchanged and the Top10 even increases slightly by 32. To better understand this phenomenon, we manually investigate the results of the variant *w/o Module Context*. We find that the module-level knowledge exerts an important influence on CosFL from two aspects: On the one hand, module-level software knowledge prompts LLM to generate queries that better match the semantics of the faulty functionality, which helps CosFL rank the buggy methods at a higher position and leads to a huge difference on the Top1 metric; On the other hand, the richer information provided by module knowledge increases the certainty of CosFL, allowing it to generate more specific queries. The reduction in ambiguity somewhat narrows the search scope, decreasing the likelihood of a small fraction of bugs being localized within Top10. From rows 2 to 3, we can see that both module- and chunk-granularity retrieval enhance the effectiveness of CosFL, contributing respectively to Top1 improvements of 12 and 19, and Top10 improvements of 9 and 23. Notably, the chunk-level retrieval plays a more critical role, offering an MRR contribution of 1.02, in contrast to 0.40 from module retrieval. This indicates that subtle variations in the detailed chunks between different methods can serve as a valid basis for determining the fault location.

Table 3. Ablation study results.

Project	Bugs	Variants	Top1	Top5	Top10	MFR	MAR
Overall	835	w/o Module Context	244 ↓ 80	531 ↓ 1	598 ↑ 32	14.38 ↑ 0.23	15.15 ↑ 0.18
		w/o Module Retrieval	312 ↓ 12	519 ↓ 13	557 ↓ 9	14.68 ↑ 0.53	15.37 ↑ 0.40
		w/o Chunk Retrieval	305 ↓ 19	509 ↓ 23	543 ↓ 23	15.15 ↑ 1.00	15.99 ↑ 1.02
		Default CosFL	324	532	566	14.15	14.97

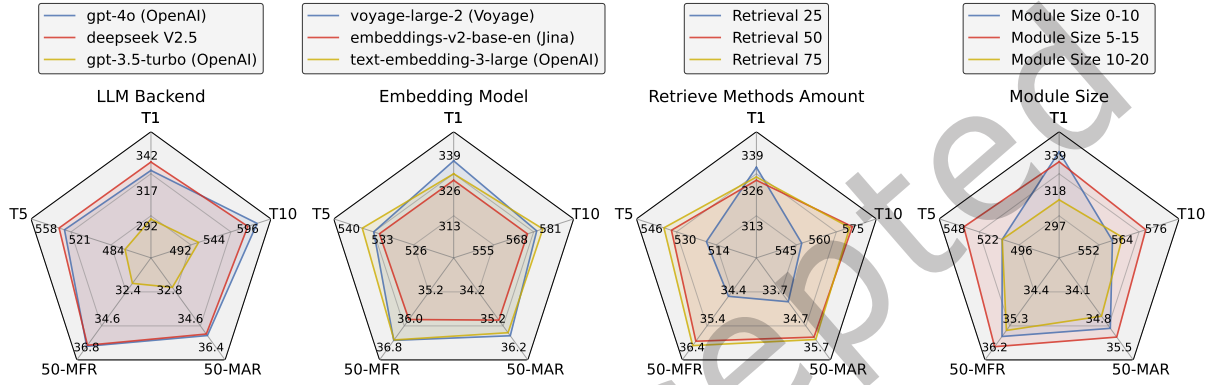


Fig. 8. The impact of different hyper-parameters on CosFL.

Answer to RQ2: All components have contributed to the fault localization performance of CosFL, among which the module-level knowledge has exerted a relatively more significant impact.

5.3 RQ3: Sensitivity Analysis

Figure 8 illustrates the performance of CosFL under different hyper-parameter settings, where the red line represents the default settings. Regarding the **LLM Backend**, we observe that LLMs with varying capabilities significantly impact CosFL's performance. For instance, CosFL (gpt-3.5-turbo) ranked 290 and 471 bugs in the Top1 and Top5, respectively. However, with the more powerful LLM gpt-4o, the Top1 and Top5 dramatically increased by 29 and 56. For LLMs with similar capacities, such as deepseek V2.5 and gpt-4o, the performance variation of CosFL is negligible. Concerning the **Embedding Model**, we notice that CosFL's performance remained stable across different series of embedding models (i.e., Jina [21], Voyage [52], and OpenAI [36]). The performance differences introduced by *text-embedding-3-large* and *voyage-large-2* did not exceed 2% across all metrics. For **Retrieve Methods Amount**, the performance of CosFL improves as the retrieved methods amount λ increases from 25 to 50, with gains of 14 and 19 in Top5 and Top10, respectively. However, as λ further increases from 50 to 75, the performance nearly plateaus. This indicates that setting λ at a moderate level is enough to fully harness CosFL's capabilities. Finally, we investigated the performance of CosFL under different **Module Size** parameters. The results demonstrate that both larger (module size ranges from 10 to 20) and smaller (0 to 10) module sizes somewhat diminished CosFL's performance. This suggests that extracting module-level knowledge of moderate size is important to fully leverage CosFL's potential, which further confirms the rationality of the Module Size Optimization operation we introduced in Section 3.1.

Table 4. Results of the query quality evaluation.

Embedding Similarity						
	Granularity	R_{best}	R_{worst}	O_{max}	O_{min}	S_{max}
Individual *	Module	3.85	7.32	0.8014	0.7835	0.8099
	Method	4.13	7.32	0.8237	0.8058	0.8393
	Chunk	4.39	7.83	0.8378	0.8176	0.8545
Grouped †	Module	3.24	6.42	0.8045	0.7842	0.8115
	Method	3.07	6.81	0.8255	0.8046	0.8434
	Chunk	3.31	7.14	0.8412	0.8190	0.8566
Human Analysis						
	Granularity	Total	Complete	Concise	Relevant	
Individual *	Module	144	94	126	113	
	Method		137	122	115	
	Chunk		140	125	108	
Grouped †	Module	86	54	77	70	
	Method		80	74	73	
	Chunk		83	76	72	
LLM Judgment						
	Granularity	Total	Complete	Concise	Relevant	
Individual *	Module	144	9	143	132	
	Method		107	143	126	
	Chunk		143	116	118	
Grouped †	Module	86	9	86	82	
	Method		68	85	77	
	Chunk		86	76	75	

† Denotes the results grouped by bug (a bug may correspond to multiple queries).

* Denotes the results for individual queries.

Answer to RQ3: CosFL shows stable performance under backend models of the same capability levels. By setting appropriate retrieve methods amount, the potential of CosFL can be further unleashed.

5.4 RQ4: Query Quality

The Query Generation (Section 3.2) phase in CosFL is critical, where the LLM analyzes root causes with the knowledge provided by the Software Knowledge Base and generates multi-granularity queries to search for buggy methods. In this RQ, we aim to perform a comprehensive evaluation of the quality of the queries generated by CosFL. Based on a 95% confidence level and a 10% confidence interval [1], we randomly sample 86 out of 835 bugs as the sample set for this research question. Since a single bug can trigger multiple failed tests and CosFL would generate three queries in different granularities (i.e., module-, method-, and chunk-level) for each failed test, the number of queries at each level in this RQ is 144. We comprehensively evaluate the query quality through three evaluation approaches, including Embedding Similarity, Human Analysis, and LLM Judgment:

Embedding Similarity. Embedding models can map textual data into a continuous vector space, thereby enabling the objective measurement of semantic similarity between texts by calculating the distances between their corresponding vectors. Intuitively, a better query should be more semantically similar to the buggy program element. For each bug, we use jina-embedding-v2-base-en as the embedding model to vectorize the query generated by CosFL and the source code of the buggy methods (Note that a single bug may be caused by multiple buggy methods). We then assess query quality by measuring the cosine similarity [44] between these embeddings. Given an embedded query $q \in \mathbb{R}^E$ for a bug (where M is the dimension of an embedding), the embedded buggy methods matrix $M^q \in \mathbb{R}^{N \times E}$ (where N is the number of buggy methods for the bug), and the cosine similarity vector for the query $s^q \in \mathbb{R}^N$ (where $s_i^q = (qM^T)_i / \|q\| \|M_i\|$), we design the following five metrics:

- O_{max} : the average max Q-Buggy Scores (i.e., the similarity scores between each query and the buggy methods), calculated by $O_{max} = \sum_{k=1}^{144} \max(s^{q_k})$. This metric evaluates the maximum semantic similarity between the query and buggy methods. The larger the O_{max} , the better the query quality.

Queries	Buggy Methods	Human Assigned Labels
Module-Level The bug likely resides in the MathUtils class, specifically within the equals method.	Method 1 <pre>public static boolean equals(double x, double y) { return (Double.isNaN(x) && Double.isNaN(y)) x == y; }</pre>	Module-Level: Concise Relevant
Method-Level The equals method is not correctly handling the comparison of arrays containing NaN values.	Method 2 ...	Method-Level: Complete Concise Relevant
Chunk-Level The buggy code may be using a direct equality check (==) for NaN values, which always returns false. The correct approach should be to use Double.isNaN() to check for NaN values.	...	Chunk-Level: Complete Concise Relevant

Fig. 9. An example questionnaire for human query quality evaluation. Given the *Queries* and the *Buggy Methods*, the participants are required to measure the queries on three criteria and assign three kinds of labels (i.e., Complete, Concise, and Relevant) to each query in *Human Assigned Labels*.

- O_{min} : the average min Q-Buggy Scores. The larger the O_{min} , the better the query quality.
- S_{max} : the average max Q-All Scores (i.e., the similarity scores between each query and all of the methods in the codebase). This metric can be used to investigate whether a query is more similar to buggy methods than other methods in the code repository.
- R_{best} : the average rank of the max Q-buggy Scores among all Q-All Scores. This metric can be used to illustrate the potential best code search performance of a query. The smaller the R_{best} , the better the query quality.
- R_{worst} : the average rank of the min Q-buggy Scores among all Q-All Scores. The smaller the R_{worst} , the better the query quality.

The result is shown in Table 4 under the subtitle Embedding Similarity. Note that since CosFL may generate multiple queries for a bug, we also show the results grouped by bug to evaluate the comprehensive performance of query quality in the FL process. Overall, we observed that the queries generated by CosFL exhibit significantly higher semantic similarity to the buggy methods compared to their similarity with other methods in the codebase. When grouped by bug, method-level queries can rank buggy methods within the range of 3.07 to 6.81 among hundreds of candidate methods. For individual queries, we found that as the query granularity goes finer (ranging from Module, Method, to Chunk), the similarity score progressively increases (the O_{max} rises from 0.8014 to 0.8237 to 0.8378) but the ranking of buggy methods correspondingly decreases (the R_{best} drops from 3.85 to 4.13 to 4.39). This phenomenon is attributed to the characteristics of queries at different granularities. Specifically, finer-grained queries provide a more detailed description of the possible problematic functionality, enhancing their semantic similarity to all methods in the codebase. Nonetheless, this increase in overall similarity may reduce the particularity of the query's semantic association with the buggy methods, thereby leading to a decline in relative ranking.

Human Analysis. Based on the existing bug explanation and query evaluation researches [23, 60], we evaluate the query quality on the following three criteria:

- Complete: the query contains a detailed description of the possible faulty functionality, rather than simply explaining the error message.
- Concise: the query succinctly describes the possible faulty functionality, without extraneous content.
- Relevant: the possible faulty functionality described in the query semantically matches at least one of the buggy methods.

We recruited six graduate students with more than 5 years of programming experience as participants. For each bug, the participant is asked to manually label the query with the above criteria by filling out the questionnaire in the format of Figure 9. We present the result in Table 4 under the subtitle Human Analysis.

Prompt 4: Query Quality Evaluation

Context

You will be presented with three levels of query text describing potential faulty functionalities in a software:

- Module Query: Indicates which modules or classes may contain faulty code.
- Method Query: Refers to specific methods that might be buggy.
- Chunk Query: Describes the detailed logic that might cause the bug.

Task

Given a list of Buggy Methods (i.e., source code of methods with known bugs), evaluate the quality of each Query and annotate each with zero, one, or more of the following labels:

- Complete: The query provides a detailed description of potentially faulty functionality, not just a restatement of an error message.
- Concise: The query expresses the faulty functionality clearly and without unnecessary content.
- Relevant: The described functionality in the query semantically matches at least one of the Buggy Methods.

Output Format

Your final output must be in JSON format, for example:

```
{
  "module_query": ["Complete", "Concise", "Relevant"],
  "method_query": ["Concise", "Relevant"],
  "chunk_query": []
}
```

Input

Module Query:<Module-Level Query Text>

Method Query:<Method-Level Query Text>

Chunk Query:<Chunk-Level Query Text>

Buggy Methods:<Source Code of the Buggy Methods>

Overall, we find that all three granularities of queries possess high relevance with the buggy methods. For example, the participants labeled about 80% (115 out of 144) of the method-level queries as Relevant. For the result grouped by bug, the participants ensured that at least one relevant method-level query is generated for 85% (73 out of 86) of the bugs. This result confirms the effectiveness of CosFL in generating high-quality queries. In terms of the Concise criteria, we observe that most of the queries were deemed concise by the participants, highlighting the strong capability of LLMs in generating queries without redundant information. Regarding the Complete criterion, we notice that the number of queries marked as Complete gradually decreases as the granularity changes from fine-grained to coarse-grained. For individual results, the number of queries labeled as Complete for chunk-, method-, and module-level granularity are 140, 137, and 94, respectively. This phenomenon aligns with the intended design of queries at different levels. As shown in Figure 9, module-level queries typically describe potentially problematic classes or functional modules. Therefore, these queries often fail to meet the Complete criterion in “*containing a detailed description of the possible faulty functionality*”. In contrast, method-level and chunk-level queries tend to provide a more detailed description of the erroneous functionality, making them more likely to be categorized as Complete.

LLM Judgment. To mitigate the influence of subjectivity in manual evaluation, we further employed an LLM to assess the quality of the queries. Specifically, we adopted the same criteria used in the human analysis and classified the queries using the gemini-2.5-flash-preview reasoning model. The prompt template for the LLM evaluator is shown in Prompt 4. We present the result in Table 4 under the subtitle LLM Judgment. Overall, the evaluation results of LLM are consistent with human analysis, as most of the queries are semantically related to at least one buggy method. For instance, 88% (126 out of 144) of method-level queries were labeled as “Relevant” by the LLM. For the result grouped by bug, CosFL is capable of generating at least one useful method-level query for 90% (77 out of 86) of the bugs. Interestingly, we observed that while the overall trends align with human

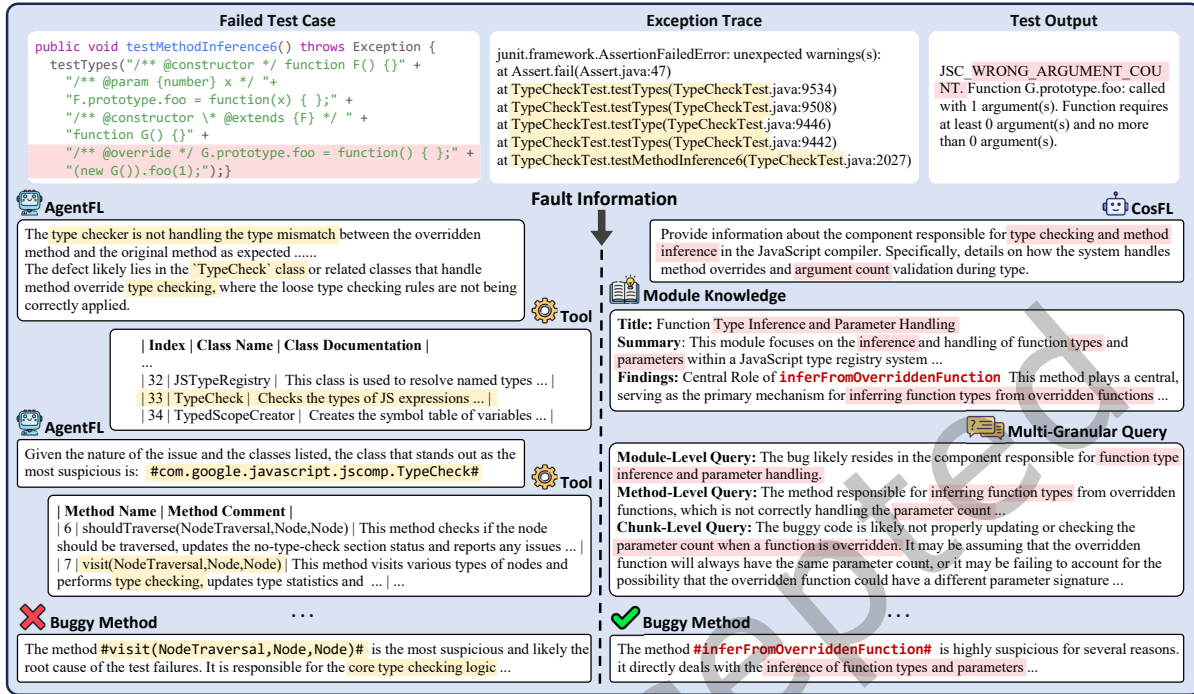


Fig. 10. Comparison between AgentFL and CosFL in localizing bug Closure-41. The keywords that AgentFL and CosFL focus on are marked in gold and red, respectively.

analysis results, the LLM appears to be more “rigorous” for the Complete criteria and more “lenient” for the Concise criteria. For instance, LLMs only categorize 6% (9 out of 144) of the module-level queries as Complete, yet almost all of them (143 out of 144) are labeled as Concise.

Answer to RQ4: The queries generated by CosFL have a high semantic relevance to the buggy code. The human analysis shows that about 80% of the method-level queries are semantically relevant to the buggy methods; for 85% of the bugs, at least one useful query is generated.

6 DISCUSSION

6.1 Case Study

To help understand how the workflow of CosFL differs from other FL approaches, we further demonstrate two specific cases. In the first case, we compare CosFL with AgentFL [39] to demonstrate the CosFL’s ability to inspect the internal logic of the software by requesting module-level knowledge. In the second case, the comparison between CosFL and AutoFL [23] indicates the superiority of semantic code search in expanding search scope.

Case 1: Better Fault Understanding. In Figure 10, we illustrate the different workflow of AgentFL and CosFL in localizing the bug Closure-41.² The cause of this bug is that the software throws an unexpected exception when performing a type check on the overridden function *G.prototype.foo*. To fix this issue, the developers modified the *inferFromOverriddenFunction* method within the *FunctionTypeBuilder* class, adding logic to enhance the parameter

²<https://storage.googleapis.com/google-code-archive/v2/code.google.com/closure-compiler/issues/issue-368.json>

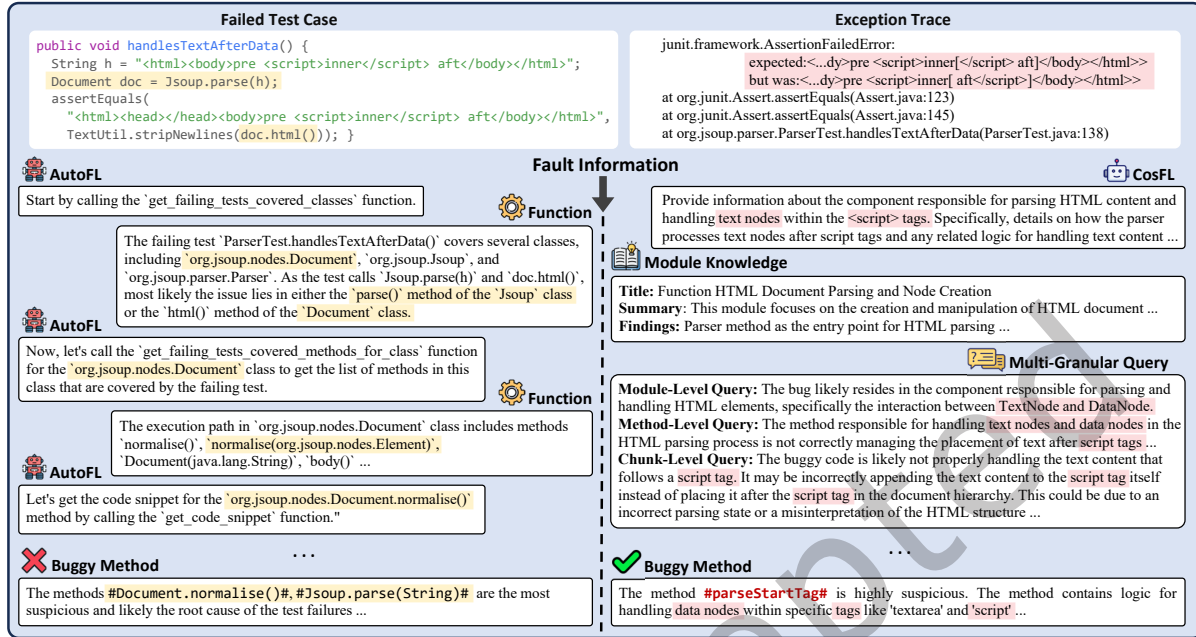


Fig. 11. Comparison between AutoFL and CosFL in localizing bug Jsoup-2. The keywords that AutoFL and CosFL focus on are marked in gold and red, respectively.

checking of the function. In this instance, since AgentFL can only start from the provided *Fault Information*, it is restricted to focusing on the *TypeCheckTest* class which repeatedly appears in the *Exception Trace*, leading to the erroneous inference that “The defect likely lies in the *TypeCheck* class.” This inference governs the follow-up localization process, misleading AgentFL to identify an unexpected method as buggy. Conversely, to gain a deeper understanding of the root cause, CosFL starts with asking “provide information about the component responsible for type checking and method inference” to obtain relevant module knowledge. The returned *Module Knowledge* highlights that the method *inferFromOverriddenFunction* “serves as the primary mechanism for inferring function types from overridden functions”. This discovery guides CosFL to the accurate fault location.

Case 2: Larger Search Scope. In Figure 11, we compare the different behaviors of AutoFL and CosFL in localizing the bug Jsoup-2³. The issue is caused by the routine that greedily parses the data content not cleaning up the stack. To fix this, the developers changed the *parseStartTag* method within class *org.jsoup.parser.Parser*. In this case, we observe that the failure of AutoFL originates from its selection of suspicious classes. Specifically, during the first *Function* call, AutoFL assesses that “most likely the issue lies in either the *Jsoup* class or *Document* class”, which results in the actual buggy class *Parser* being overlooked. Subsequently, as indicated in the third dialogue box on the left side of the figure, AutoFL further requests to “get the list of methods in *Document* class that are covered by the failing test”, exacerbating the error and ultimately preventing AutoFL from identifying the expected buggy method. In contrast, when referring to CosFL, we observe that although *Module Knowledge* did not provide any useful context in this instance, semantic code search played a crucial role. By focusing on software functionalities related to “data node” and “script tag”, CosFL successfully matched the buggy method

³<https://github.com/jhy/jsoup/issues/22>

parseStartTag from the whole code base through code search, as the method “*contains logic for handling data nodes within specific tags*”.

6.2 Comparison with Traditional FL Techniques

We also compared CosFL with other traditional FL techniques. These contain the spectrum-based technique Ochiai [2] and various learning-based FL techniques, including FLUCCS [46] based on machine learning, DeepFL [28] based on deep learning, and GRACE [34] based on graph neural networks. Particularly, to align with practical application scenarios, we focused on the cross-project performance of the learning-based approaches (i.e., trained on the Defects4J V1.2.0 benchmark [22] and tested on V2.0.0). As shown in Table 5, CosFL ranked 105 buggy methods in the first position, which is 73 more than Ochiai, 48 more than FLUCCS, 43 more than DeepFL, and 20 more than GRACE. On the Top3 and Top5 metrics, CosFL also consistently outperforms other FL approaches. This indicates the substantial potential of employing semantic code search for fault localization.

Table 5. Results on Defects4J V2.0.0.

# Bugs	Techniques	Top1	Top3	Top5
280	Ochiai	32	74	93
	FLUCCS	57	97	119
	DeepFL	43	89	112
	GRACE	85	119	140
	CosFL	105	137	178

6.3 Cost Analysis

Time and API usage costs are critical concerns for practitioners when using LLM-based FL systems. Therefore, we recorded the execution time and API usage of CosFL at each step, the results are shown in Figure 12. Overall, CosFL spends an average of 144.5 seconds and \$0.076 to locate a bug. As a comparison, existing research [7] found that practitioners spend an average of 32 minutes to diagnose a bug, which indicates that applying CosFL would significantly help developers save their time costs. Particularly, we observed that the cost incurred during the knowledge base construction phase (i.e., Module-Level, Method & Chunk Level, and Embedding) is relatively high, taking an average of 126.3 seconds and \$0.074 for a bug. In contrast, during the fault localization process (i.e., Query Generation and Fault Retrieval), CosFL requires only an average of 12.7 seconds and \$0.0022 to find a fault location. It is worth noting that in practical software development scenarios, we do not need to rebuild the knowledge base from scratch each time; instead, we can incrementally update its content along with the software life cycle. Therefore, we consider the relatively high cost of the initial knowledge base construction to be acceptable.

6.4 Threats to Validity

Internal. The main internal threat comes from the data leakage problem. The knowledge about the Defects4J [22] and the GrowingBugs [19] benchmark may have been included in the LLM training corpus. To mitigate this threat as much as possible, we ensure that the input provided to the LLM does not contain any project name, bug ID, or human-written bug report related to a particular bug. Additionally, we conduct a fair comparison between CosFL and other LLM-based FL techniques using the same LLM backend on the identical benchmark. The evaluation results demonstrate the superiority of our approach.

External. Our evaluation of CosFL primarily focuses on software projects with Java unit tests. Our findings may not be generalizable to other programming languages or different levels of testing. To mitigate this threat, we adhered to the principle of scalability when implementing CosFL. Specifically, we adopted a common software hierarchy division (e.g., Module, Method, and Chunk) in the design of CosFL, which ensures that CosFL can be easily extended to other object-oriented programming languages such as Python and C++.

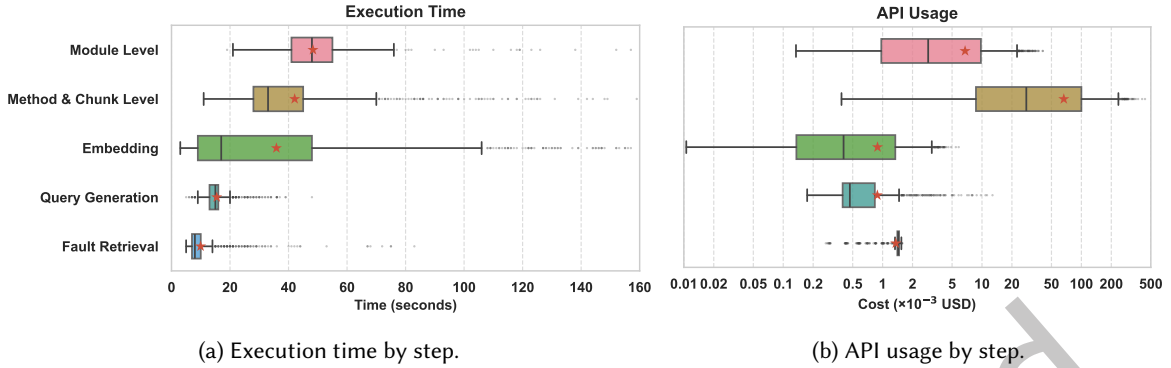


Fig. 12. Cost analysis results. Module Level, Method & Chunk Level, and Embedding are the three stages of Software Knowledge Base Construction in Fig. 3.

6.5 Limitation and Future Work

While CosFL demonstrates promising initial results, it is important to acknowledge several limitations that require further investigation.

Reliance on Query Quality. The effectiveness of CosFL heavily relies on the quality of the queries generated by the LLM. The LLM’s ability to accurately understand the error message and formulate relevant search queries is crucial. Factors such as the complexity of the root cause, as well as the LLM’s training data and inherent biases, could impact the quality of the generated queries, leading to suboptimal localization in some cases. To address this, future research can focus on generating better queries. This could involve exploring different prompting techniques, fine-tuning the LLM on the FL task, or incorporating a context collection agent to guide the LLM in creating more precise and effective queries.

The Accuracy of Code Search Techniques. Our current approach assumes that the top results returned by the code search are indeed the buggy methods. However, in complex software systems, a large number of program elements may exhibit functional similarities, which makes it difficult to ensure that the retrieved methods are directly associated with the actual faulty location. To improve the reliability of our method, future efforts should focus on enhancing the effectiveness of the code search. This could involve utilizing rerank techniques [62], employing static analysis techniques to confirm the presence of the error in the identified code, or incorporating an LLM to further confirm the relevance between the queries and the suspicious methods.

Handling of Complex and Distributed Errors. Our method currently focuses on localizing buggy methods. However, many software errors might involve multiple interacting components or be distributed across several methods, which our current approach may struggle to pinpoint. Future work could aim at extending our approach to complex error scenarios. This might involve analyzing relationships between methods, tracking data flow, or incorporating techniques like program slicing to identify the interconnected components involved in a bug.

Support for more programming languages. CosFL currently focuses on fault localization for Java projects. There are two primary reasons: (1) Toolchain Support for Program Analysis: Our approach involves program instrumentation and the construction of dynamic method call graphs. The Java ecosystem offers a more extensive toolchain for these specific types of program analysis. This supports a more straightforward implementation and reliable data collection for our methodology in its current stage of development. (2) Object-Oriented Characteristics: Java’s design as a statically-typed, object-oriented language aligns well with the “multi-granularity code search” aspect of our method. While Python is also object-oriented, Java’s stricter adherence to certain object-oriented principles and its more explicit class and method structures provided a clearer framework for

evaluating our approach. In our future work, we plan to extend our evaluation to include projects developed in other languages, such as Python, to assess the generalizability and potential adaptations required for CosFL.

7 RELATED WORK

7.1 Early Fault Localization Techniques

Fault localization (FL) is a crucial component of software debugging and has been extensively studied. To help developers automatically find the fault locations in the entire code base, early FL techniques including Spectrum-based FL (SBFL) [2, 3, 57], Mutation-based FL (MBFL) [58], and Information Retrieval-based FL (IRFL) [54].

SBFL identifies potentially faulty program elements by assuming that the fault locations are more frequently executed by failing test cases than by passing ones. To implement SBFL, coverage data is gathered for each program element e , specifically noting how many passing tests $T_p(e)$ and failing tests $T_f(e)$ execute each element. Various formulas are then applied to determine the suspiciousness scores that rank these elements accordingly. Some of the commonly used formulas include Ochiai [2], Jaccard [3], and DStar [57].

MBFL aims to enhance FL techniques by integrating impact information. Given that code elements executed by passing or failing tests might not influence the test results, MBFL utilizes mutation testing to mimic the impact of individual code elements for more accurate fault identification. Metallaxis [37], the first generalized MBFL method, is predicated on the intuition that if a mutant affects failing tests (i.e., the test result changes from fail to pass after mutation), the corresponding code element might be responsible for the test failures. Conversely, if a mutant affects passing tests, the associated code element is less likely to be faulty. Metallaxis then employs traditional SBFL formulas to determine the suspiciousness of each mutant. The highest suspiciousness score among mutants is then assigned as the suspiciousness of the related code element.

IRFL has emerged within the realm of fault location by leveraging a few selected bug reports. One notable method is BugLocator introduced by Zhou *et al.* [65], which employs an information retrieval approach to identify files pertinent to bug fixes. This method enhances the VSM model by adjusting the ranks based on the information from previously fixed similar bugs. Saha *et al.* [42] present BLUIR, which discovers that structured information retrieval based on code constructs can significantly improve FL effectiveness.

Early FL technologies had certain limitations. Specifically, SBFL and MBFL typically calculate the suspiciousness of fault locations through statistical analysis, without considering the semantics of the program itself. The IRFL largely depend on bug reports, which are not always of high quality or readily available in practice [54]. Notably, we differentiate our approach from IRFL as CosFL does not rely on bug reports and can understand semantic information more accurately through LLMs.

7.2 Learning-based Fault Localization Techniques

With the advancement of artificial intelligence, the learning-based fault localization (LBFL) methods have emerged. These methods embed code into vector space through feature engineering and representation learning, enabling neural networks to identify incoming bugs by fitting them with situations during training.

In general, most LBFL methods focus on utilizing coverage data more comprehensively and incorporate additional valuable information such as code history and bug reports. FLUCCS [46] enhances SBFL by integrating code and change metrics. DeepFL [28] leverages Recurrent Neural Networks (RNN) [49] and Multi-Layer Perceptrons (MLP) [38] to synthesize knowledge from four different dimensions: SBFL, mutation-based fault localization, code complexity, and text similarity. GRACE [34] combines coverage data with detailed code structures using a Graph Neural Network (GNN) [31] model. More LBFL techniques include CNNFL [64], TraPT [29], CombineFL [66], etc.

However, due to the lack of scalability and interpretability, these approaches struggle to be applied in practical scenarios [25]. In Section 6.2, we compared the performance differences between the CosFL and LBFL methods under the cross-project scenario and the results demonstrate the superiority of our approach.

7.3 LLM-based Fault Localization Systems

Recently, the powerful code analysis capabilities of LLMs have offered new opportunities for FL, with preliminary attempts such as AutoFL [23] and AgentFL [39] have guided LLMs to autonomously navigate through the code base and identify defect locations.

AutoFL [23] is an automated FL system that leverages the LLMs to not only pinpoint potential bug locations but also to provide detailed explanations on the nature of the bug. AutoFL enables LLMs to effectively navigate the source code by allowing them to execute specific functions that retrieve information about covered classes, covered methods, and their corresponding implementations and documentation. With the provided functions, the LLM systematically makes function calls to gather the necessary information and deduce the buggy location, the natural language response output by LLMs is then used as the explanation.

AgentFL [39] is an LLM-based FL system designed for project-level bug localization in large software codebases. It utilizes multiple LLM-driven agents within a structured three-stage workflow: (1) Fault Comprehension. This stage analyzes the fault's potential causes by capturing comprehensive test execution behaviors through lightweight program instrumentation. It provides richer fault insights compared to methods that rely solely on error messages and failed test cases. (2) Codebase Navigation. This stage systematically narrows the search space by identifying suspicious classes and methods. It combines existing and LLM-enhanced documentation to link potential fault causes to specific code elements effectively. (3) Fault Confirmation. In this stage, the LLMs iteratively review and score suspicious methods identified earlier, prioritizing them to determine the most likely faulty method.

In Section 5.1, we comprehensively compared CosFL with AutoFL and AgentFL. The results show that CosFL significantly outperforms other LLM-based FL systems, which reveals the potential of boosting FL from the perspective of semantic code search.

7.4 Semantic Code Representation for Fault Localization

Some existing works have also attempted to leverage semantic code representations for fault localization. For example, FixLocator [30] is a Deep Learning (DL)-based fault localization approach that supports the detection of faulty statements within one or multiple methods requiring modification in the same fix (i.e., co-change (CC) fixing locations for a fault). HMCBL [11] proposed a directed, multiple-label code graph representation named Semantic Flow Graph (SFG) to capture code semantics and subsequently introduced a hierarchical momentum contrastive bug localization technique. However, we argue that these existing methods differ from our proposed approach in two key aspects:

Conceptual Differences. CosFL emphasizes the idea of using code search for fault localization. Code search is a technique that utilizes a query (often expressed in natural language) to find semantically matching code snippets within a code repository. Thus, a significant and challenging part of our work is the generation of high-quality natural language queries. The other approaches based on code representation learning, while also aiming to understand code, focus more on building models that learn to recognize buggy patterns directly from code features, making them less directly aligned with the concept of utilizing code search for FL.

Methodological Differences. The FL techniques based on representation learning typically involve designing and training specialized neural network architectures directly on labeled datasets of code and bugs. Nonetheless, the rapid evolution of LLMs has triggered a reshaping of the entire fault localization paradigm. Under this background, we deeply integrate the capabilities of LLMs and embedding models in this work to explore potential directions for the development of FL within this new technological landscape.

8 CONCLUSION

In this work, we present CosFL, a novel Fault Localization (FL) approach inspired by semantic Code Search (CS). CosFL proposes to regard FL tasks as a two-step process: the *query generation* for generating a natural language query to describe the problematic software functionalities, and the *fault retrieval* which employs CS to directly match buggy program elements from the entire code repository. Evaluation results on 835 real bugs from 23 Java projects show that CosFL can localize 324 bugs within Top-1 and 532 bugs within Top-5, which significantly outperforms other state-of-the-art FL approaches. Through the ablation study and sensitivity analysis, we further verify the contributions of various components within CosFL and its robustness under different parameter settings. Finally, we dissect the internal workflow of CosFL and demonstrate its advantages in the FL task with two specific case studies. Overall, this paper thoroughly explores the feasibility of boosting FL from the perspective of CS, potentially paving new pathways for the advancement of FL techniques.

9 DATA AVAILABILITY

The code of CosFL is publicly available at <https://github.com/IntHelloWorld/CosFL>.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (Grant No.62402506, No.62474196) as well as the Research Foundation from NUDT (Grant No. ZK24-05).

REFERENCES

- [1] 2024. *Sample size calculator*. <https://www.surveysystem.com/sscalc.htm>
- [2] Rui Abreu, Peter Zoetewij, and Arjan J.c. Van Gemund. 2006. An Evaluation of Similarity Coefficients for Software Fault Localization. In *2006 12th Pacific Rim International Symposium on Dependable Computing (PRDC'06)*. 39–46. <https://doi.org/10.1109/PRDC.2006.18>
- [3] Rui Abreu, Peter Zoetewij, and Arjan JC Van Gemund. 2007. On the accuracy of spectrum-based fault localization. In *Testing: Academic and Industrial Conference Practice and Research Techniques-MUTATION*. IEEE, 89–98. <https://doi.org/10.1109/TAIC.PART.2007.13>
- [4] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL].
- [5] Toufique Ahmed and Premkumar Devanbu. 2023. Few-Shot Training LLMs for Project-Specific Code-Summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (Rochester, MI, USA) (ASE '22)*. Association for Computing Machinery, New York, NY, USA, Article 177, 5 pages. <https://doi.org/10.1145/3551349.3559555>
- [6] Punam Bedi and Chhavi Sharma. 2016. Community detection in social networks. *WIREs Data Mining and Knowledge Discovery* 6, 3 (2016), 115–135. <https://doi.org/10.1002/widm.1178>
- [7] Marcel Böhme, Ezekiel O Soremekun, Sudipta Chattopadhyay, Emamurho Ugherughe, and Andreas Zeller. 2017. Where is the bug and how is it fixed? an experiment with practitioners. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering*. ACM, 117–128. <https://doi.org/10.1145/3106237.3106255>
- [8] Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Hongzhi Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, Hui Liu, and Jiliang Tang. 2024. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *SIGKDD Explor. Newsl.* 25, 2 (March 2024), 42–61. <https://doi.org/10.1145/3655103.3655110>
- [9] ChromaDB. 2024. ChromaDB. <https://github.com/chroma-core/chroma>.
- [10] DeepSeek. 2024. DeepSeek-V2.5-1210. <https://huggingface.co/deepseek-ai/DeepSeek-V2.5-1210>.
- [11] Yali Du and Zhongxing Yu. 2023. Pre-training Code Representation with Semantic Flow Graph for Effective Bug Localization. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (San Francisco, CA, USA) (ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 579–591. <https://doi.org/10.1145/3611643.3616338>
- [12] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. arXiv:2404.16130 [cs.CL] (2024).
- [13] Peter Emerson. 2013. The original Borda count and partial voting. *Social Choice and Welfare* 40, 2 (2013), 353–358. <https://doi.org/10.1007/s00355-011-0603-9>
- [14] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. 2018. Deep code search. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 933–944. <https://doi.org/10.1145/3180155.3180167>

- [15] Martin Hitz and Behzad Montazeri. 1995. Measuring coupling and cohesion in object-oriented systems. In *Proc. International Symposium on Applied Corporate Computing*, 1995.
- [16] Francesca Incitti, Federico Urli, and Lauro Snidaro. 2023. Beyond word embeddings: A survey. *Information Fusion* 89 (2023), 418–436. <https://doi.org/10.1016/j.inffus.2022.08.024>
- [17] JavaAPI. 2024. instrument. <https://docs.oracle.com/javase/8/docs/api/java/lang/instrument/package-summary.html>.
- [18] JGraphT. 2024. JGraphT. <https://jgraph.org>.
- [19] Yanjie Jiang, Hui Liu, Xiaoqing Luo, Zhihao Zhu, Xiaye Chi, Nan Niu, Yuxia Zhang, Yamin Hu, Pan Bian, and Lu Zhang. 2022. Bugbuilder: An automated approach to building bug repository. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1443–1463. <https://doi.org/10.1109/TSE.2022.3177713>
- [20] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *arXiv:2408.02479 [cs.SE]*.
- [21] JinaAI. 2024. Jina Embeddings. <https://jina.ai/embeddings>.
- [22] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis*. ACM, 437–440. <https://doi.org/10.1145/2610384.2628055>
- [23] Sungmin Kang, Gabin An, and Shin Yoo. 2024. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. *Proc. ACM Softw. Eng.* 1, FSE, Article 64 (July 2024), 23 pages. <https://doi.org/10.1145/3660771>
- [24] Irvin R. Katz and John R. Anderson. 1987. Debugging: An Analysis of Bug-Location Strategies. *Human-Computer Interaction* 3, 4 (1987), 351–399. https://doi.org/10.1207/s15327051hci0304_2
- [25] Pavneet Singh Kochhar, Xin Xia, David Lo, and Shanping Li. 2016. Practitioners’ expectations on automated fault localization. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (Saarbrücken, Germany) (ISSTA 2016)*. Association for Computing Machinery, New York, NY, USA, 165–176. <https://doi.org/10.1145/2931037.2931051>
- [26] Tim Korson and John D. McGregor. 1990. Understanding object-oriented: a unifying paradigm. *Commun. ACM* 33, 9 (Sept. 1990), 40–60. <https://doi.org/10.1145/83880.84459>
- [27] A. Güneş Koru, Dongsong Zhang, Khaled El Emam, and Hongfang Liu. 2009. An Investigation into the Functional Form of the Size-Defect Relationship for Software Modules. *IEEE Transactions on Software Engineering* 35, 2 (2009), 293–304. <https://doi.org/10.1109/TSE.2008.90>
- [28] Xia Li, Wei Li, Yuqun Zhang, and Lingming Zhang. 2019. DeepFL: Integrating multiple fault diagnosis dimensions for deep fault localization. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 169–180. <https://doi.org/10.1145/3293882.3330574>
- [29] Xia Li and Lingming Zhang. 2017. Transforming Programs and Tests in Tandem for Fault Localization. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 92 (oct 2017), 30 pages.
- [30] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2022. Fault localization to detect co-change fixing locations. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 659–671. <https://doi.org/10.1145/3540250.3549137>
- [31] Yujia Li, Richard Zemel, Marc Brockschmidt, and Daniel Tarlow. 2016. Gated Graph Sequence Neural Networks. In *Proceedings of ICLR’16 (proceedings of iclr’16 ed.)*.
- [32] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (02 2024), 157–173. https://doi.org/10.1162/tacl_a_00638
- [33] LlamaIndex. 2024. LlamaIndex. https://github.com/run-llama/llama_index.
- [34] Yiling Lou, Qihao Zhu, Jinhao Dong, Xia Li, Zeyu Sun, Dan Hao, Lu Zhang, and Lingming Zhang. 2021. Boosting Coverage-Based Fault Localization via Graph-Based Representation Learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 664–676. <https://doi.org/10.1145/3468264.3468580>
- [35] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an LLM to Help With Code Understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE ’24)*. Association for Computing Machinery, New York, NY, USA, Article 97, 13 pages. <https://doi.org/10.1145/3597503.3639187>
- [36] OpenAI. 2024. OpenAI. <https://openai.com>.
- [37] Mike Papadakis and Yves Le Traon. 2015. Metallaxis-FL: mutation-based fault localization. *Software Testing, Verification and Reliability* 25, 5-7 (2015), 605–628.
- [38] Marius-Constantin Popescu, Valentina E. Balas, Liliana Perescu-Popescu, and Nikos Mastorakis. 2009. Multilayer Perceptron and Neural Networks. *WSEAS Trans. Cir. and Sys.* 8, 7 (jul 2009), 579–588.
- [39] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2024. AgentFL: Scaling LLM-based Fault Localization to Project-Level Context. *arXiv:2403.16362 [cs.SE]* (2024).

- [40] Sofia Reis, Rui Abreu, and Marcelo D’Amorim. 2019. Demystifying the Combination of Dynamic Slicing and Spectrum-Based Fault Localization. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (Macao, China) (IJCAI’19)*. AAAI Press, 4760–4766.
- [41] Chanchal K Roy, James R Cordy, and Rainer Koschke. 2009. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming* 74, 7 (2009), 470–495. <https://doi.org/10.1016/j.scico.2009.02.007>
- [42] Ripon K. Saha, Matthew Lease, Sarfraz Khurshid, and Dewayne E. Perry. 2013. Improving bug localization using structured information retrieval. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 345–355. <https://doi.org/10.1109/ASE.2013.6693093>
- [43] Ensheng Shi, Yanlin Wang, Wenchao Gu, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. CoCoSoDa: Effective Contrastive Learning for Code Search. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 2198–2210. <https://doi.org/10.1109/ICSE48619.2023.00185>
- [44] Pinky Sitikhu, Kritish Pahi, Pujan Thapa, and Subarna Shakya. 2019. A Comparison of Semantic Similarity Methods for Maximum Human Interpretability. In *2019 Artificial Intelligence for Transforming Business and Society (AITB)*, Vol. 1. 1–4.
- [45] Victor Sobreira, Thomas Durieux, Fernanda Madeiral, Martin Monperrus, and Marcelo de Almeida Maia. 2018. Dissection of a bug dataset: Anatomy of 395 patches from Defects4J. In *Proceedings of the 25th International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 130–140. <https://doi.org/10.1109/SANER.2018.8330203>
- [46] Jeongju Sohn and Shin Yoo. 2017. FLUCCS: Using Code and Change Metrics to Improve Fault Localization. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (Santa Barbara, CA, USA) (ISSTA 2017)*. Association for Computing Machinery, New York, NY, USA, 273–283. <https://doi.org/10.1145/3092703.3092717>
- [47] Weisong Sun, Chunrong Fang, Yifei Ge, Yuling Hu, Yuchen Chen, Qianjun Zhang, Xiuting Ge, Yang Liu, and Zhenyu Chen. 2024. A Survey of Source Code Search: A 3-Dimensional Perspective. *ACM Trans. Softw. Eng. Methodol.* 33, 6, Article 166 (June 2024), 51 pages. <https://doi.org/10.1145/3656341>
- [48] Tan Thongtan and Tanasanee Phienthrakul. 2019. Sentiment Classification Using Document Embeddings Trained with Cosine Similarity. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: Student Research Workshop*, Fernando Alva-Manchego, Eunsol Choi, and Daniel Khoshabi (Eds.). Association for Computational Linguistics, Florence, Italy, 407–414. <https://doi.org/10.18653/v1/P19-2057>
- [49] Mikolov Tomáš, Karafiát Martin, Burget Lukáš, Černocký Jan, and Khudanpur Sanjeev. 2010. Recurrent neural network based language model. *Interspeech 2010* (09 2010), 1045.
- [50] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific reports* 9, 1 (2019), 1–12. <https://doi.org/10.1038/s41598-019-41695-z>
- [51] treesitter. 2024. tree-sitter. <https://tree-sitter.github.io/tree-sitter>.
- [52] VoyageAI. 2024. VoyageAI. <https://www.voyageai.com/>.
- [53] Yao Wan, Zhangqian Bi, Yang He, Jianguo Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Hai Jin, and Philip Yu. 2024. Deep Learning for Code Intelligence: Survey, Benchmark and Toolkit. *ACM Comput. Surv.* 56, 12, Article 309 (Oct. 2024), 41 pages. <https://doi.org/10.1145/3664597>
- [54] Qianqian Wang, Chris Parnin, and Alessandro Orso. 2015. Evaluating the usefulness of IR-based fault localization techniques. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (Baltimore, MD, USA) (ISSTA 2015)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/2771783.2771797>
- [55] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F Bissyandé, and Xiaoguang Mao. 2024. Fusing Code Searchers. *IEEE Transactions on Software Engineering* (2024).
- [56] Shangwen Wang, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Yan Lei, and Xiaoguang Mao. 2023. Two Birds with One Stone: Boosting Code Generation and Code Search via a Generative Adversarial Network. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 239 (oct 2023), 30 pages. <https://doi.org/10.1145/3622815>
- [57] W Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2013. The DStar method for effective software fault localization. *IEEE Transactions on Reliability* 63, 1 (2013), 290–308.
- [58] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740. <https://doi.org/10.1109/TSE.2016.2521368>
- [59] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, Franz Wotawa, and Dongcheng Li. 2023. *Software Fault Localization: an Overview of Research, Techniques, and Tools*. John Wiley & Sons, Ltd, Chapter 1, 1–117. <https://doi.org/10.1002/9781119880929.ch1>
- [60] Wan-Ching Wu, Diane Kelly, and Kun Huang. 2012. User evaluation of query quality. In *Proceedings of the 35th International ACM SIGIR Conference on Research and Development in Information Retrieval (Portland, Oregon, USA) (SIGIR ’12)*. Association for Computing Machinery, New York, NY, USA, 215–224. <https://doi.org/10.1145/2348283.2348315>
- [61] Yonghao Wu, Zheng Li, Jie M. Zhang, Mike Papadakis, Mark Harman, and Yong Liu. 2023. Large Language Models in Fault Localisation. [arXiv:2308.15276 \[cs.SE\]](https://arxiv.org/abs/2308.15276).

- [62] Shicheng Xu, Liang Pang, Jun Xu, Huawei Shen, and Xueqi Cheng. 2024. List-aware Reranking-Truncation Joint Model for Search and Retrieval-augmented Generation. In *Proceedings of the ACM Web Conference 2024 (Singapore, Singapore) (WWW '24)*. Association for Computing Machinery, New York, NY, USA, 1330–1340. <https://doi.org/10.1145/3589334.3645336>
- [63] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv:2405.15793 [cs.SE]*.
- [64] Zhuo Zhang, Yan Lei, Xiaoguang Mao, and Panpan Li. 2019. CNN-FL: An effective approach for localizing faults using convolutional neural networks. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 445–455.
- [65] Jian Zhou, Hongyu Zhang, and David Lo. 2012. Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports. In *2012 34th International Conference on Software Engineering (ICSE)*. 14–24. <https://doi.org/10.1109/ICSE.2012.6227210>
- [66] Daming Zou, Jingjing Liang, Yingfei Xiong, Michael D. Ernst, and Lu Zhang. 2021. An Empirical Study of Fault Localization Families and Their Combinations. *IEEE Transactions on Software Engineering* 47, 2 (2021), 332–347. <https://doi.org/10.1109/TSE.2019.2892102>

Received 25 December 2024; revised 3 June 2025; accepted 27 July 2025